

Architectures pour les réseaux de neurones

S. Mancini



Plan

- ✖ Virgule fixe
- ☐ Réseaux de neurones
- ☐ Architecture pour les réseaux de neurones
- ☐ Exemple d'architectures spécifiques

Nombres entiers et Virgule flottante

❑ Nombre entiers:

$$a = \sum_{i=0}^{n-1} a_i 2^i$$

❑ Nombres en virgule flottante:

$$a = (-1)^s . 1, M_a . 2^{(E_a - offset)}$$

Norme IEEE 754:

	Encodage	Signe	Exposant	Mantisse	Valeur d'un nombre
Simple précision	32 bits	1 bit	8 bits	23 bits	$(-1)^S \times 1, M \times 2^{(E-127)}$
Double précision	64 bits	1 bit	11 bits	52 bits	$(-1)^S \times 1, M \times 2^{(E-1023)}$

Problématique des opérations virgule flottante

- ? Comment réaliser une **addition** entre deux nombres en virgule flottante (32 bits) ?
- ? ☐ $3 + 30$
- ? ☐ $3 + 100\,0000$
- ? ☐ $3 + 20\,000\,000$
- ? ☐ $100000000 - 99999999$

Problématique des opérations virgule flottante

? Comment réaliser une **addition** entre deux nombres en virgule flottante (32 bits) ?

? ☐ 3 + 30

? ☐ 3 + 100 0000

? ☐ 3 + 20 000 000

☐ 10000000 - 9999999

```
int main(void)
{
float a=3, b=200000000;
float c= a+b;
float e= c-b;
printf("c= %f, e= %f\n", c, e);
}
> c= 200000004.000000, e= 4.000000
```

Opérateurs virgule flottante

Les opérateurs virgule flottante, non détaillés dans ce cours, nécessitent:

- ❑ Alignement des virgules avant l'opération (sauf mul & div)
Calcul d'un exposant commun et expansion des nombres
- ❑ Opération arithmétique en nombre entier
- ❑ Recherche du 1er bit significatif
- ❑ Ré-alignement de la mantisse
- ❑ Gestion des approximations/débordement

 La norme IEEE 754 impose la gestion de valeurs spécifiques:

- Nombres dénormalisés (autour de 0)
- NaN, Infini +/-, etc ...

Opérateur matériels en virgule flottante

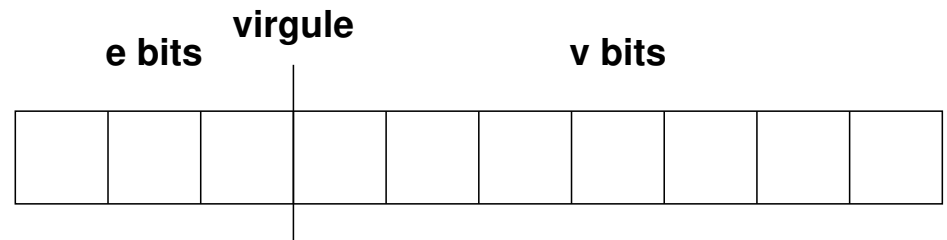
- ☐ La gestion des nombres en virgule flottante a un coût matériel très important
- Un opérateur float représente 20 à 30 fois la surface de son équivalent entier
- ☐ Un opérateur float prend du temps !
- 4 à 5 fois son équivalent entier
- La présence d'opérateurs float accroît le coût, la consommation énergétique et rend la synchronisation complexe.

? Peut on faire plus économe ?

Représentation virgule fixe

Représenter des nombres non-entier avec une partie fractionnaire fixe:

$$a = \sum_{i=-v}^{e-1} a_i 2^i$$



- ❑ La **partie entière** e : nombre de bits à gauche de la virgule
- ❑ La **partie fractionnaire** v : nombre de bits à droite de la virgule

En non-signé, on peut représenter des nombres de 0 à $2^e - 2^{-v}$ avec un pas de 2^{-v} .

? Représenter 3,64 avec le format ci-dessus

Comparaison virgule fixe/flottante

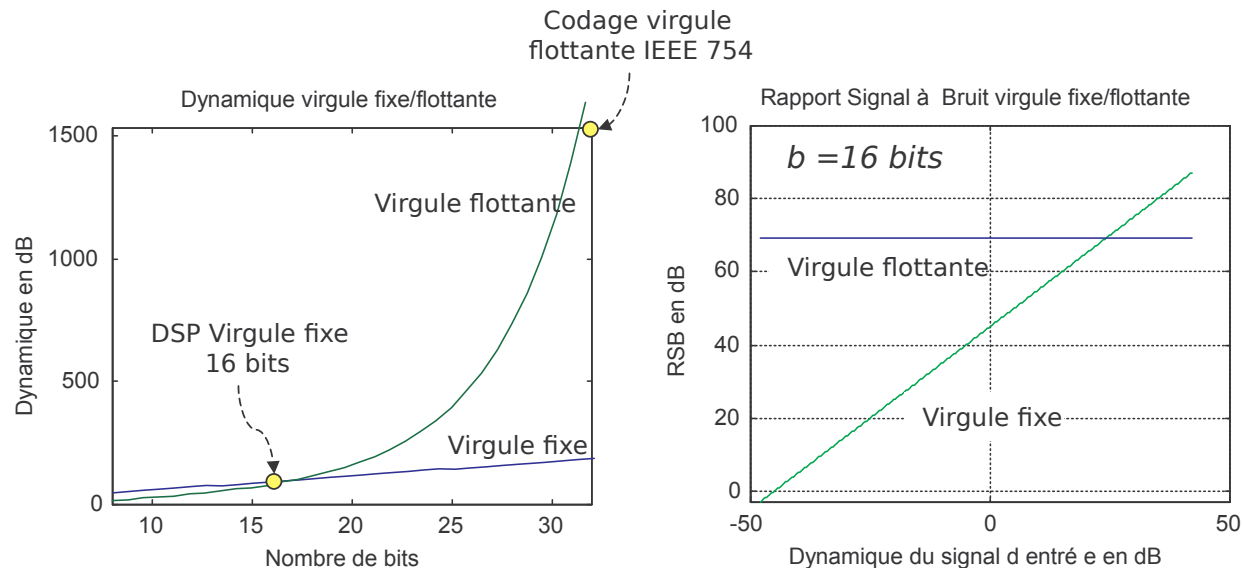
❑ Niveau de dynamique

$$D_N(dB) = 20 \log \frac{\max |x|}{\min |x|}$$

❑ Rapport Signal à Bruit de Quantification

$$\rho_{dB} = 10 \log \frac{P_s}{P_e}$$

c.f. D. Menard



Usages

❑ Arithmétique virgule flottante

- . Largeur des données stockées en mémoire : 32 bits (IEEE 754)
- . Opérateurs complexes (gestion de la mantisse et de l'exposant)
- . L'utilisateur ne doit pas coder les données ➡ Temps de développement faible

Applications spécifiques : audionumérique, faible volume

❑ Arithmétique virgule fixe

- . Opérateurs plus simples ➡ Processeur plus rapide
- . Largeur des données stockées en mémoire : environ 16 bits

➡ Consommation moins importante

➡ Processeur moins cher ➡ Applications grand public

DSP virgule fixe : 95% des ventes

Arrondi , troncature & saturation

Pour ce cours, on utilise la notation $a_{e,v}$:
la variable a est représentée sur e bits à gauche de la virgule et v bits à droite.

❑ Troncature : supprimer des bits de poids faible ou fort

$$a_{e',v'} = \lfloor a_{e,v} \rfloor_{e',v'}$$

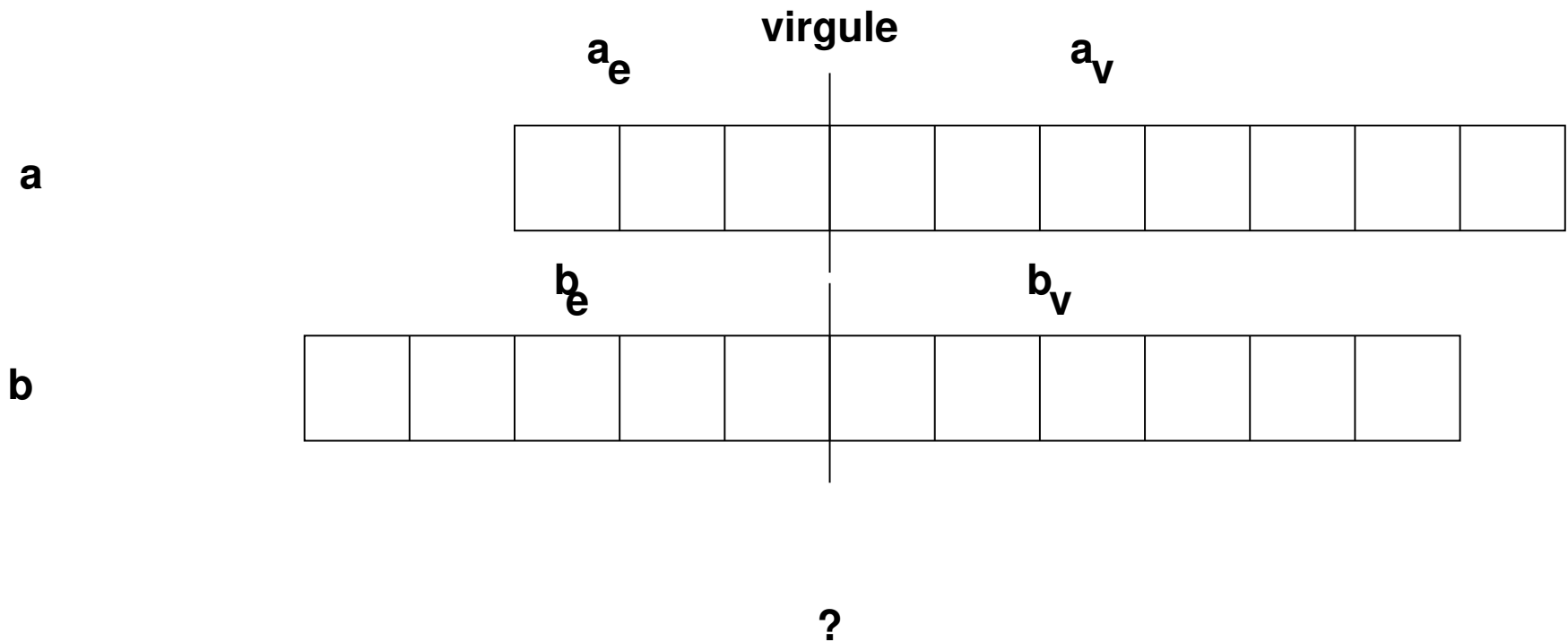
❑ Arrondi : prendre la valeur la plus proche

$$a_{e,v'} = \lfloor a_{e,v} + 2^{-v'-1} \rfloor_{e,v'}$$

❑ Saturation: supprimer des bits de poids fort sans “modulo”

$$a_{e',v'} = \lfloor \min(2^{e'} - 1, a_{e,v}) \rfloor_{e',v'}$$

Addition



Addition

Une addition/soustraction entre $a_{e,v}$ et $b_{e,v}$, produit $c_{e,v}$ tel que

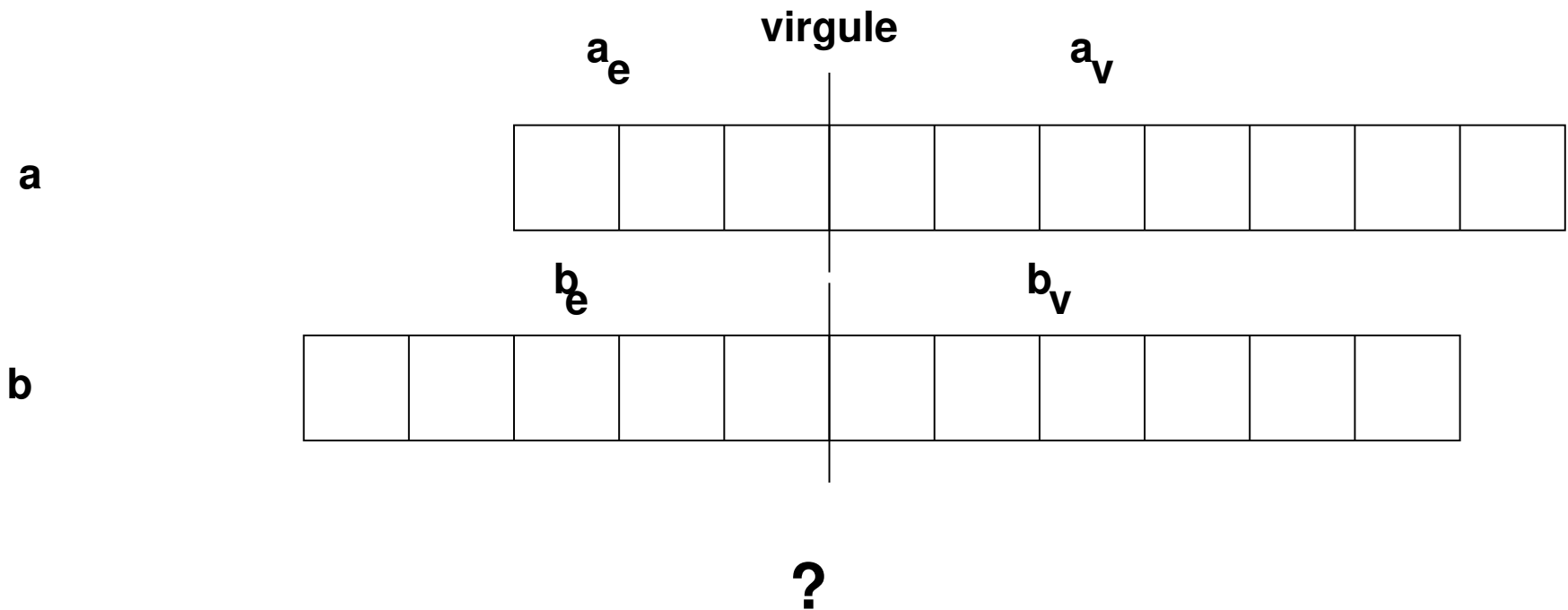
- $c_e = \max(a_e, b_e) + 1$
- $c_v = \max(a_v, b_v)$

L'opération addition/soustraction nécessite d'aligner les opérandes en décalant l'argument de plus petite partie fractionnaire.

- ❑ Logiciel: opération de décalage
- ❑ Matériel: coût nul (fils)

➡ La taille (dynamique) des opérateurs grandit au fur et à mesure des opérations ($\log(n)$)

Multiplication



Multiplication

Une multiplication entre $a_{e,v}$ et $b_{e,v}$, produit $c_{e,v}$ tel que

- $c_e = a_e + b_e$
- $c_v = a_v + b_v$

L'opération multiplication ne nécessite pas d'alignement !

➡ La taille (dynamique) des opérateurs grandit au fur et à mesure des opérations (n)

Division

La division est l'opération matérielle la plus complexe!

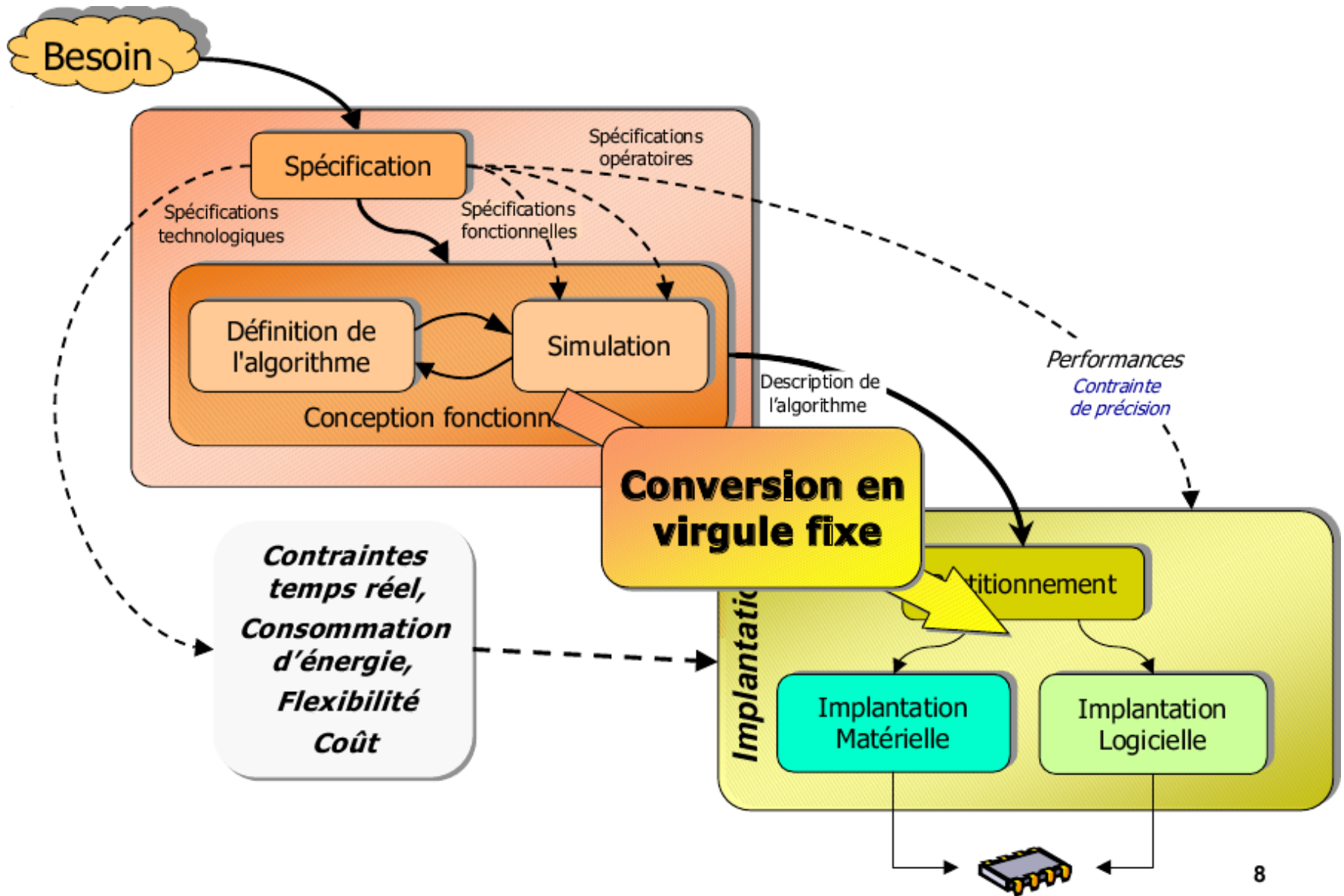
Notion de bruit de calcul

Lorsqu'un calcul nécessite plusieurs additions/multiplications successives, il n'est pas toujours possible de conserver tous les bits: les résultats intermédiaires doivent être arrondi/tronqué/saturé.

L'arrondi/troncature a le même effet qu'un bruit ajouté aux données !

 Il est nécessaire de calculer ou mesurer l'effet du passage à la virgule fixe par rapport à une application de référence !

Cycle de développement (D. Menard)



Bilan sur la virgule fixe

La virgule fixe était utilisée bien avant la virgule flottante !

Les opérateurs “virgule fixe” sont des opérateur **entiers** classiques, associés à des décalages fixes (fils) ou dynamiques.

Ils sont utilisés lorsque les grandeurs physiques manipulées sont de l'ordre du raisonnable. Les applications classiques sont les télécom, la vidéo, le traitement d'image et l'IA.

- 👍 La virgule fixe permet des **économies** de surface et d'énergie (facteur 20 à 30), et de temps de calcul.
- 👍 La conversion virgule flottante vers la virgule fixe
- 👍 s'accompagne d'une étude du **bruit de calcul** par rapport à
- 👍 une tolérance.

- ☐ Virgule fixe

- ✖ Réseaux de neurones

 - ✓ Réseaux de neurone et apprentissage

 - ☐ Convolutional Neural Network (CNN)

- ☐ Architecture pour les réseaux de neurones

- ☐ Exemple d'architectures spécifiques

Principe généraux de l'IA & réseaux de neurones

L'Intelligence Artificielle (IA) par Réseau de Neurone Artificiel (**RNA**) à appliquer sur des données un traitement lui-même paramétré par un apprentissage préalable d'après une base de données.

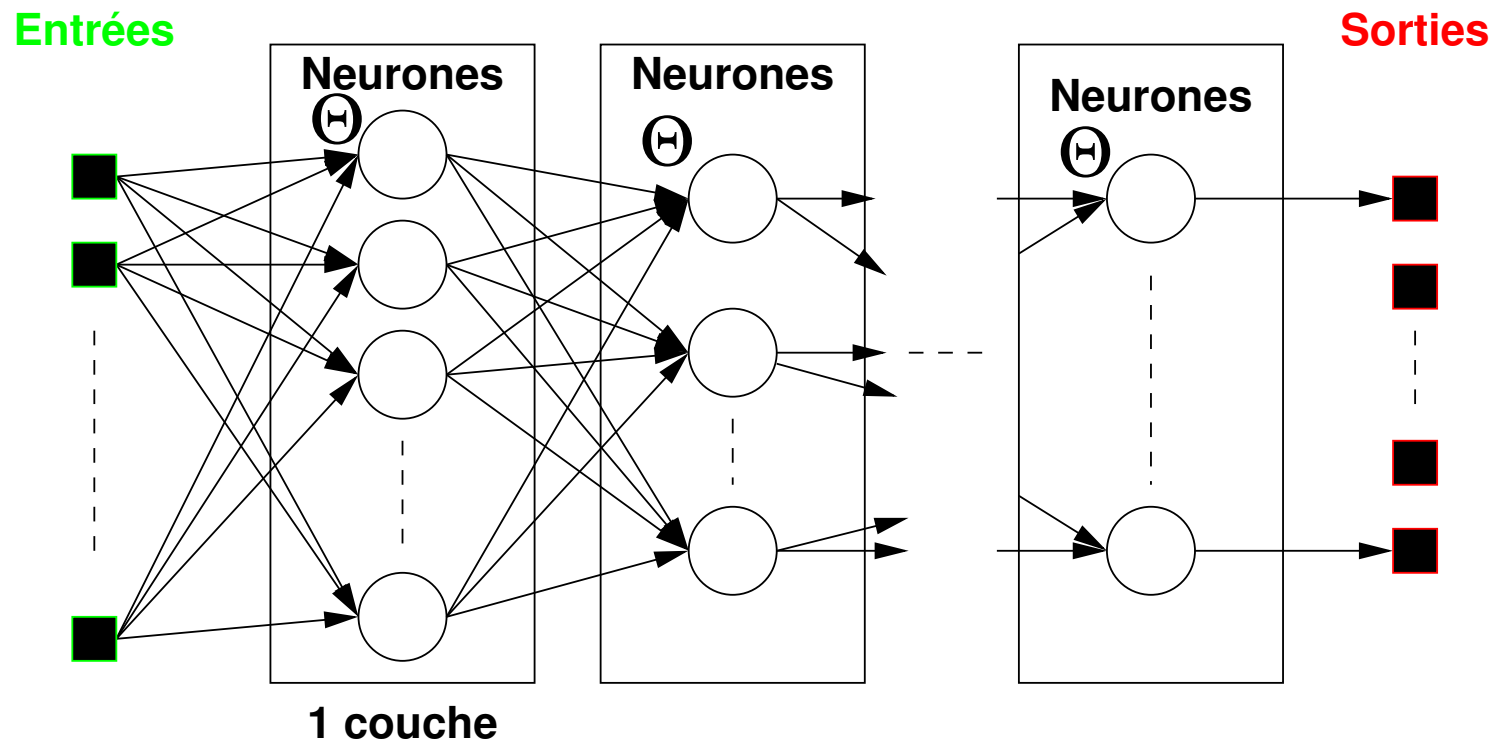
Les tâches d'IA par RNA sont essentiellement:

- ❑ **Détection** Probabilité de présence ou l'absence d'un motif
- ❑ **Classification** Probabilités d'appartenance à chaque classe
- ❑ **Régression** Estimation de la valeur d'un paramètre (taille d'un objet, ...)
- ❑ **Synthèse** Production d'images réalistes, ...

Les **Convolutional Neural Network** (CNN) sont des RNA spécifiques au traitement d'image, qui utilisent des convolutions sur des tableaux de données.

Réseaux de Neurones & CNN

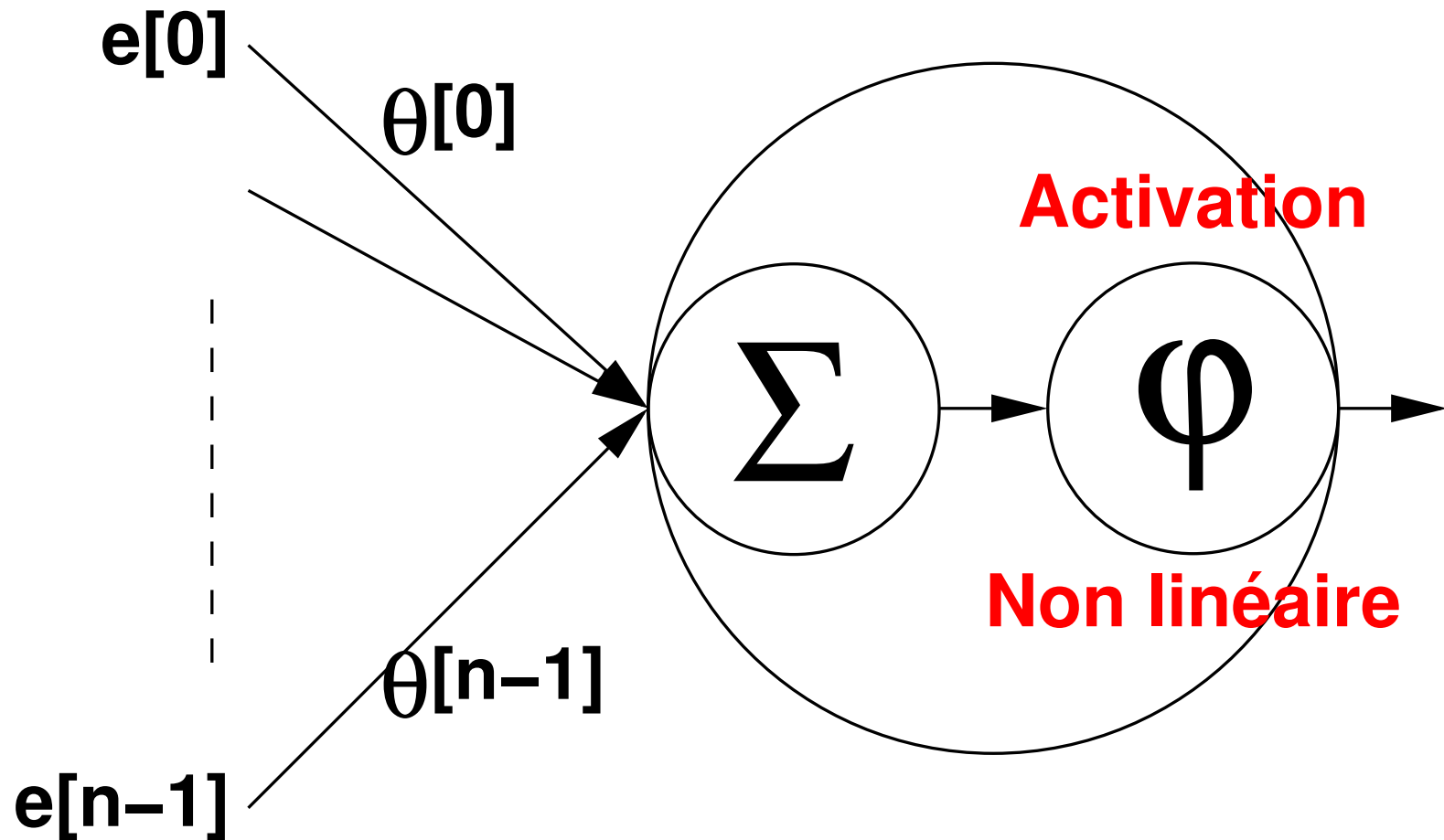
Un RNA est censé reproduire des **couches de neurones**.
Chaque couche est constituée de **sommes de produits** des entrées par des **coefficients** Θ , suivis d'une fonction d'activation. **L'apprentissage** consiste à calculer les coefficients Θ à partir d'une base de donnée.



Réseaux de Neurones & CNN

Chaque neurone est constitué d'une couche linéaire (somme de produit) et d'une fonction d'activation non-linéaire.

Un **CNN** remplace les sommes de produit par des convolutions.



Apprentissage

L'apprentissage est une tâche complexe, non traitée dans ce cours, et il n'est pas possible d'optimiser automatiquement certains paramètres du RNA:

- ❑ L'**architecture** du réseau : nombre de couches, nature des couche, tailles des couches, etc. . .
- ❑ Les **hyperparamètres** d'apprentissage: vitesse d'apprentissage, taille des lots, . . .

Pour ne pas **sur-entraîner** le réseau et le rendre trop spécifique (**généralisation**), l'apprentissage se fait en réalité sur une partition de la BDD en deux groupes de données:

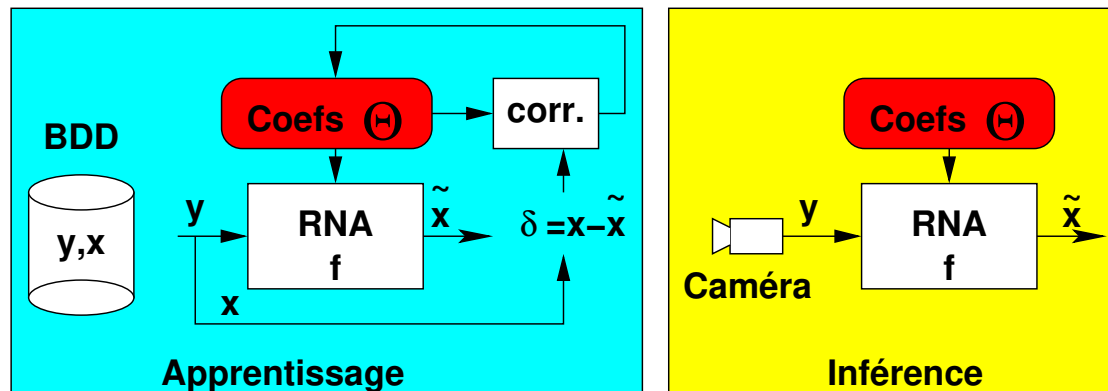
- ❑ Des **données d'apprentissage**
- ❑ Des **données de test** pour mesurer sa qualité

Fonctions génériques 1/2

Une tâche d'IA consiste à déterminer les données 'génératrices' x qui ont produit une 'observation' y (mesure).

Par ex., la donnée génératrice 'chat' génère toutes les images de chats !

Hypothèse : il existe une fonction f , inconnue à-priori, qui permet de passer d'une 'observation' y à ses données 'génératrices' $\tilde{x} = f(y)$



Le “**deep-learning**” consiste à chercher f à partir d'un ensemble d'observations préalables, la base de données d'apprentissage **étiquetée**: on connaît des données y et x associées.

Fonctions génériques 2/2

On construit f à partir de fonctions génériques paramétrables de façon systématique:

$$f(\theta, y) = f_{n-1}(\theta_{n-1}, f_{n-2}(\theta_{n-2}, \dots f_0(\theta_0, y)))$$

Les fonctions f_n sont les “**couches**” du réseau, $\theta = \{\theta_0, \dots \theta_{n-1}\}$ sont les coefficients du réseau, dont les valeurs proviennent d'un apprentissage.

Les couches d'un réseau de neurones sont typiquement:

- ❑ Fonctions **linéaires**
 - . Couches 'totalement' connectées : le perceptron
 - . Convolution
- ❑ Fonctions d'**activation** non-linéaires (RELU, sigmoïd, ...)
- ❑ **Sous-échantillonnage** (maxpool, ...)

L'apprentissage d'un réseau de neurones

L'apprentissage consiste à déterminer θ pour une base de données étiquetée (x, y) , de façon à minimiser l'erreur entre le résultat de $f(\theta, y)$ et x , l'étiquette attendue associée à y .

- ❑ Un objectif : la mission du réseau de neurones

- ❑ Une base de données d'apprentissage

Des données connues et étiquetées du résultat attendu, pour l'apprentissage des paramètres θ et le test du réseau

- ❑ Une 'architecture' de réseau

Le nombre et la nature des couches.

- ❑ Une méthode d'apprentissage

- ❑ Fonction de coût et régularisation

Pour forcer les paramètres à respecter certaines caractéristiques.

- ❑ Méthode d'optimisation

Le plus souvent descente de gradient stochastique

- Virgule fixe

- ✖ Réseaux de neurones

 - Réseaux de neurone et apprentissage

 - ✓ Convolutional Neural Network (CNN)

- Architecture pour les réseaux de neurones

- Exemple d'architectures spécifiques

Les **Convolutional Neural Network**, sont une technique de Deep Learning pour de nombreuses tâches de **traitement d'image**: classification, détection, identification, amélioration de qualité, flou optique. . .

Un CNN est constitué seulement des couches de

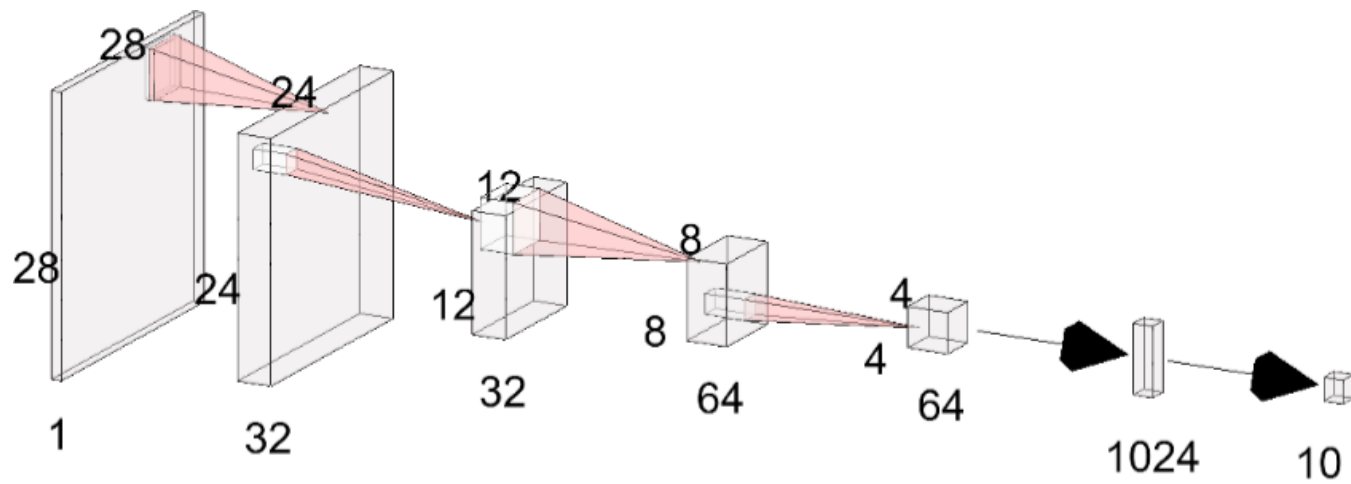
- ❑ Convolution
- ❑ RELU
- ❑ Sous-échantillonnage par "pooling"
- ❑ Complètement connecté (perceptron)

Chaque couche prend en entrée une **feature map** et produit une feature map.

Structures de données d'un CNN

Chaque couche d'un CNN prend en entrée une **feature map** et produit une feature map.

Une fmap est un tableau 3D: (Canaux * Largeur * Hauteur)



La nature des données représentées varie selon la couche:

- ❑ En entrée, l'image traitée (image RGB 3 canaux, ou monochromatique 1 canal)
- ❑ Au début, des caractéristiques 'bas niveau' (probabilité de présence d'une couleur, d'un coin, ...)
- ❑ A la fin, plus abstrait, la probabilité de présence d'un motif complexe

Convolution

Un **kernel** K de **convolution** 2D:

$$S_{i,j} = (K \times I)(i,j) = \sum_m \sum_n I_{i+m,j+n} K_{m,n}$$

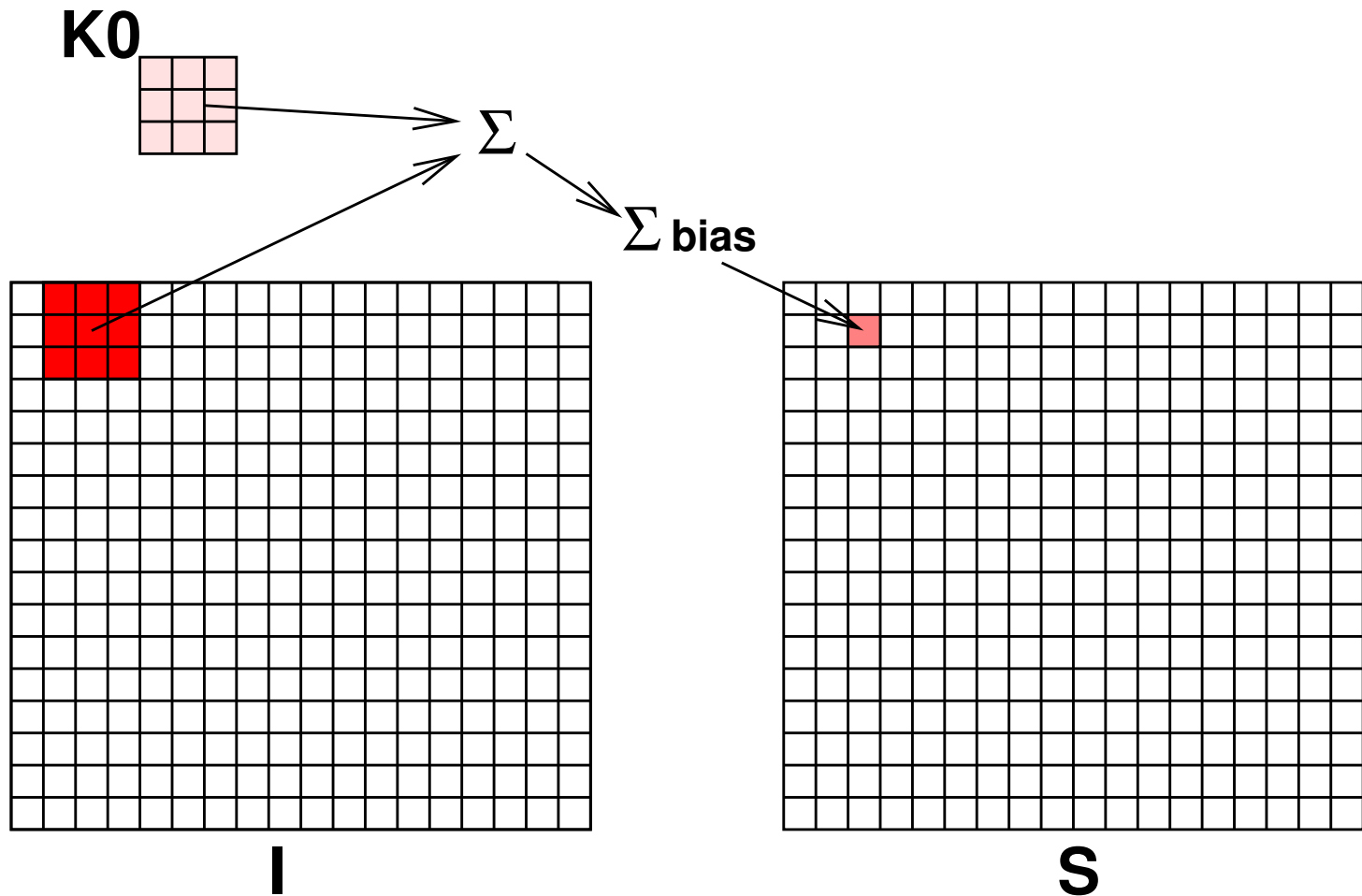
Dans un CNN, une 'feature map' est multi-canal:

$$S_{c,i,j} = \sum_{l,m,n} I_{l,i+m,j+n} K_{c,l,m,n} + \text{bias}_c$$

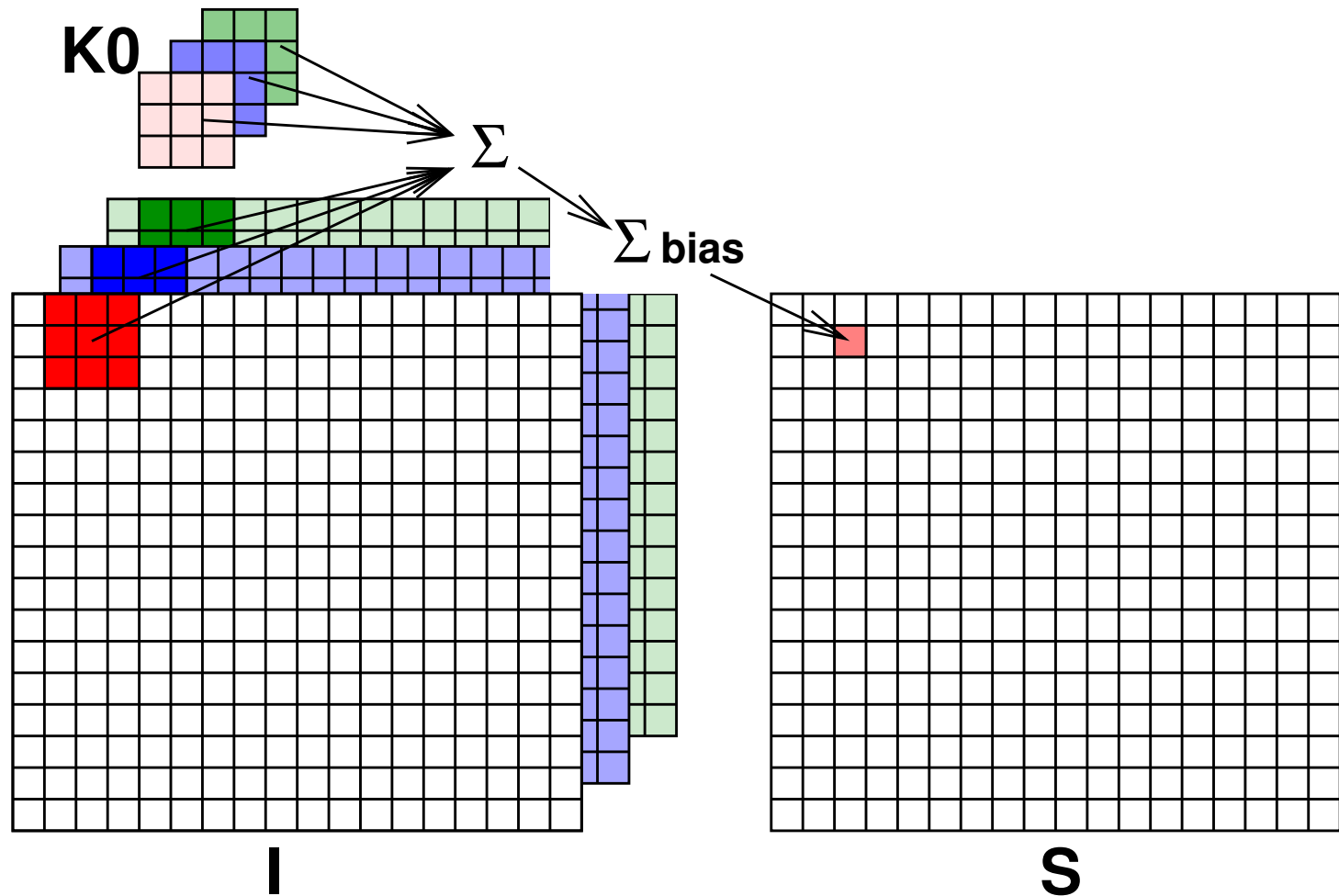
Un canal de sortie c est produit à partir de tous les canaux l de la 'feature map' d'entrée.

L'apprentissage fixe toutes les valeurs de $K_{c,l,m,n}$ pour toutes les couches.

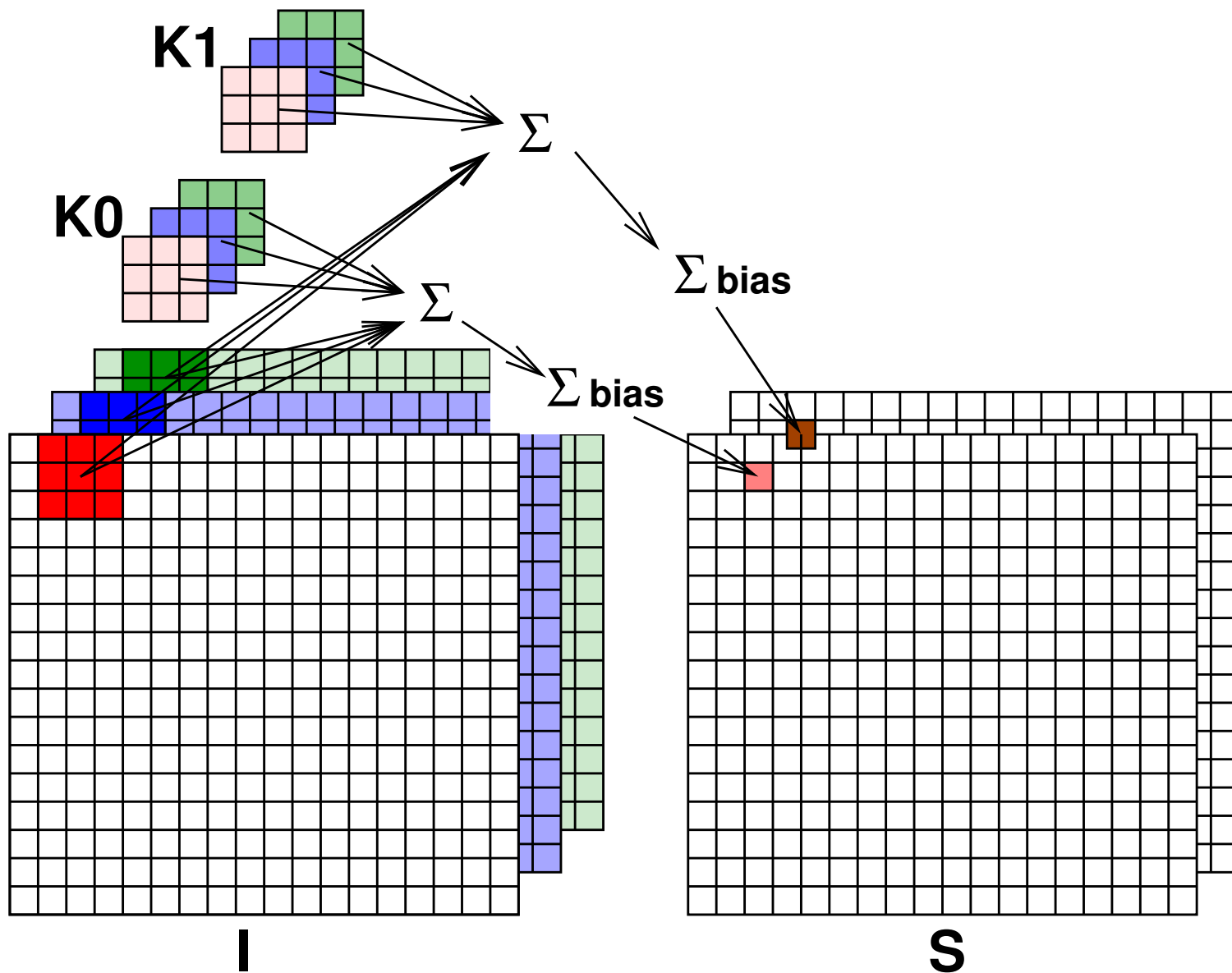
Convolution (bis)



Convolution (bis)



Convolution (bis)



RELU

RELU pour *REctified Linear Unit* est la fonction d'activation non-linéaire des CNN:

$$RELU(a) = \max(0, a)$$

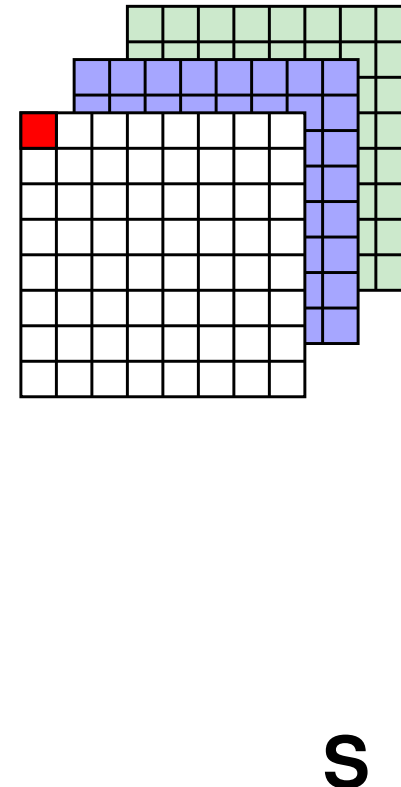
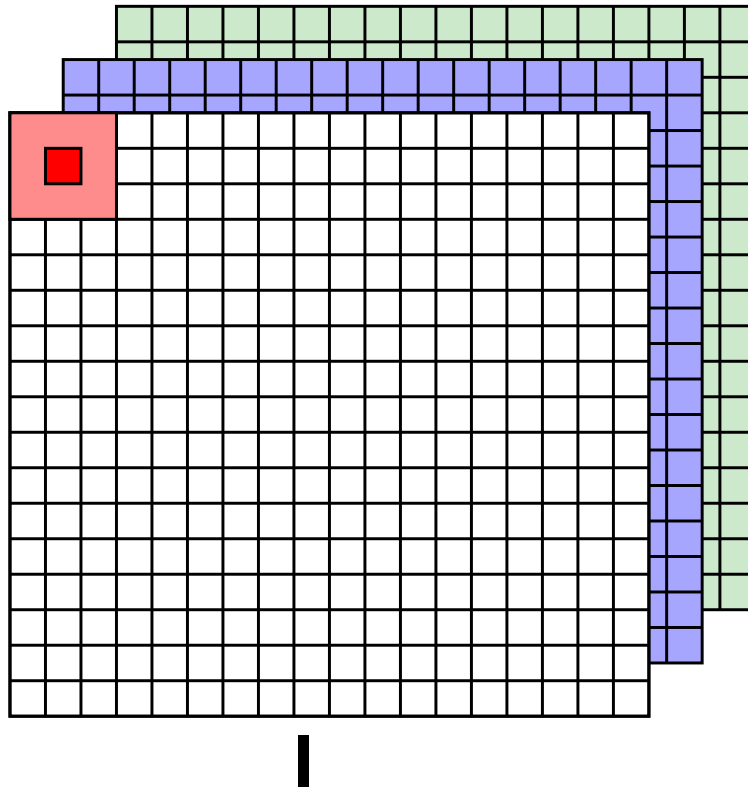
Par rapport à la sigmoïd du perceptron, les avantages de RELU sont:

- ❑ Facile à implémenter
- ❑ Dérivable (presque) partout, à dérivée non-nulle même pour les valeurs extrêmes
- ➡ L'apprentissage est facilité

MaxPool

Maxpool est l'autre fonction non-linéaire de choix:

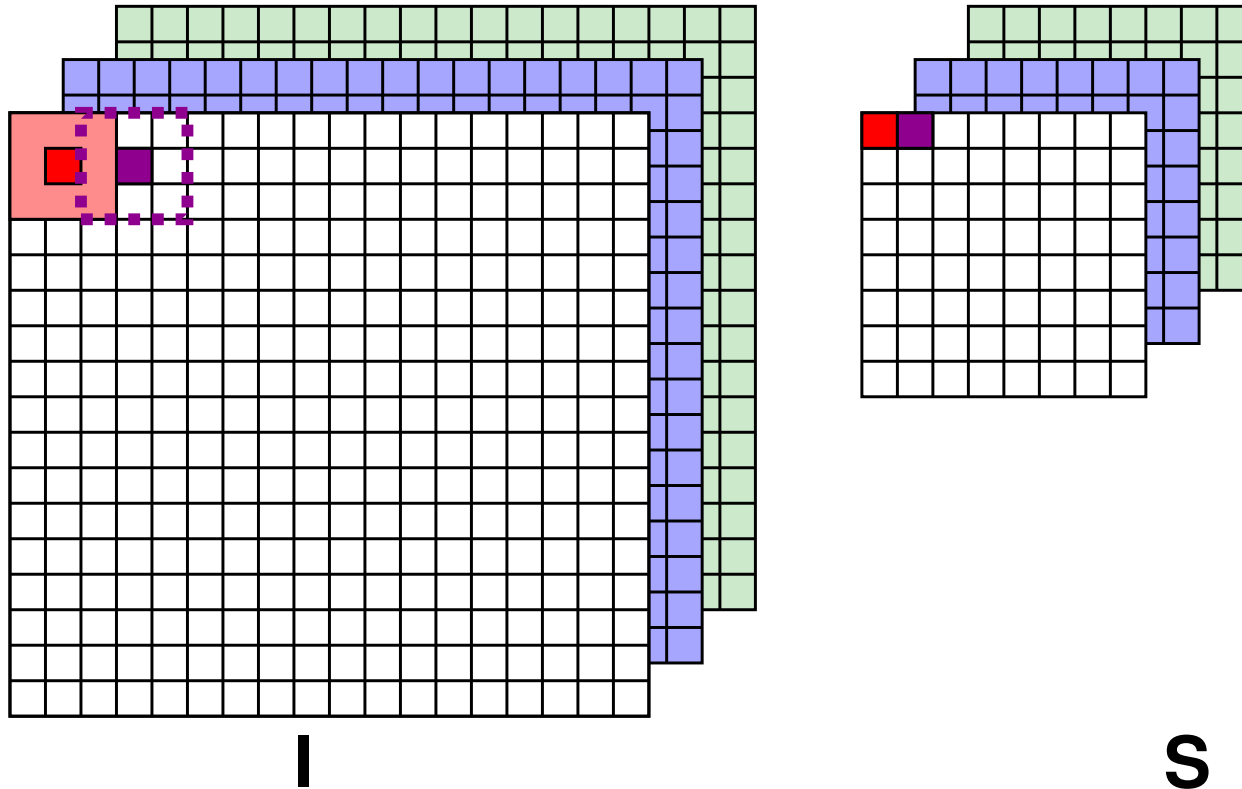
- ❑ Sous-échantillonnage pour la réduction des données
- ❑ Remplace un groupe de données par son maximum (ou médian, etc ...)



MaxPool

Maxpool est l'autre fonction non-linéaire de choix:

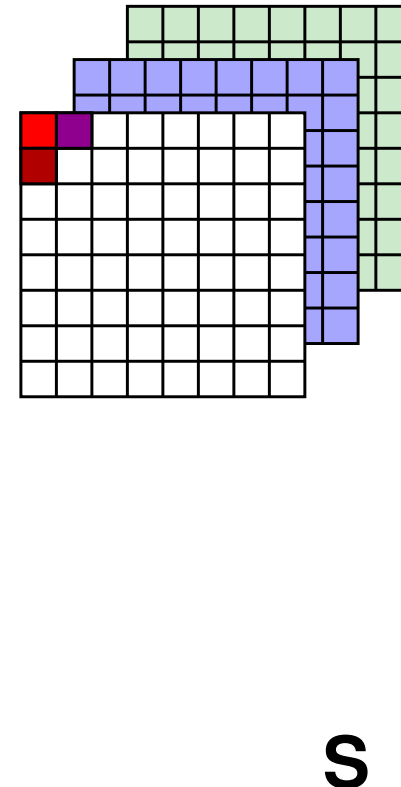
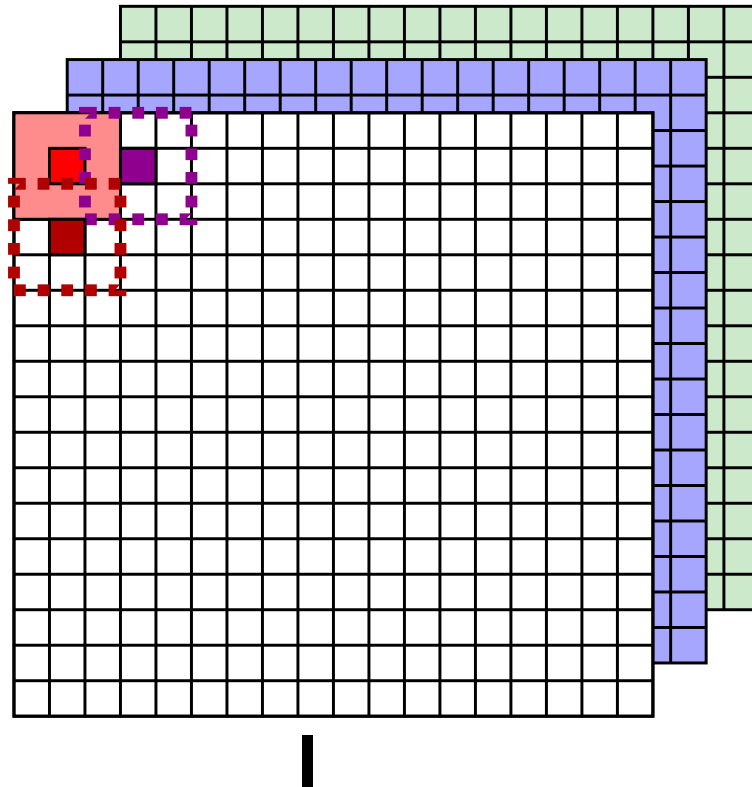
- ❑ Sous-échantillonnage pour la réduction des données
- ❑ Remplace un groupe de données par son maximum (ou médian, etc ...)



MaxPool

Maxpool est l'autre fonction non-linéaire de choix:

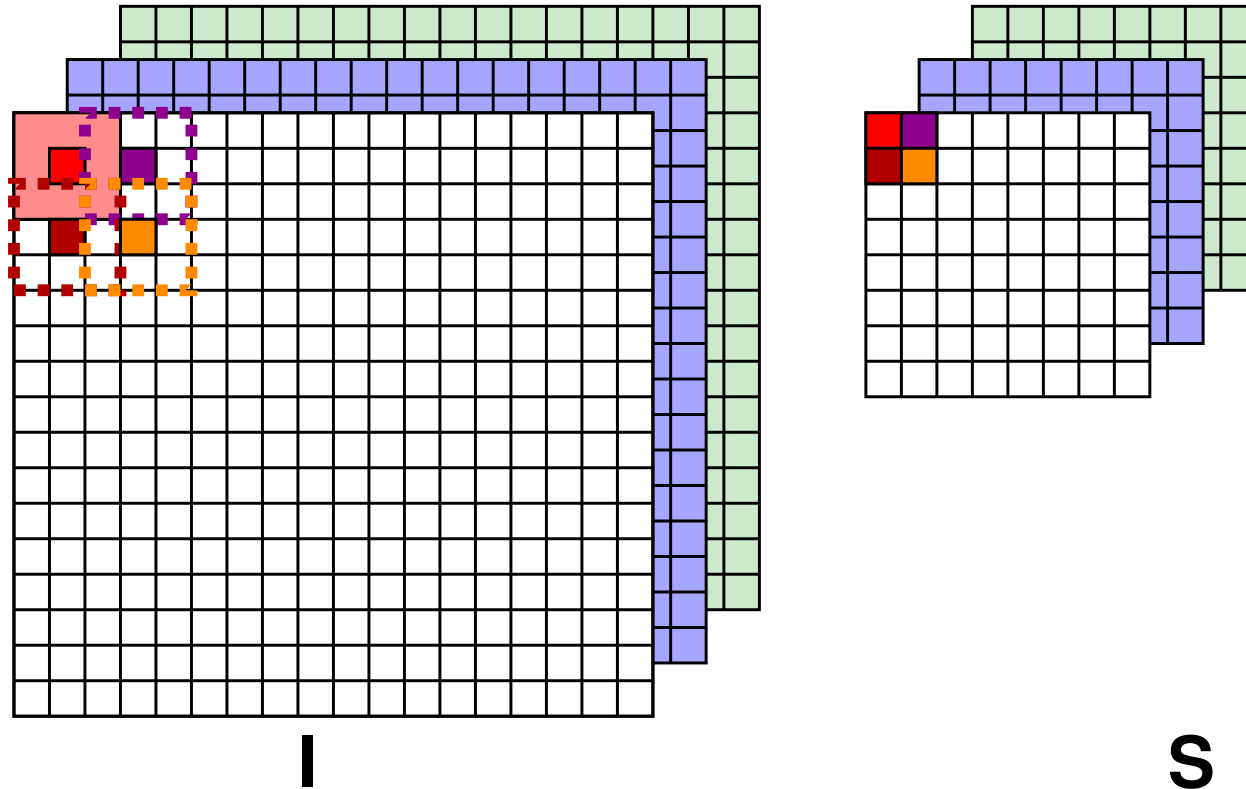
- ❑ Sous-échantillonnage pour la réduction des données
- ❑ Remplace un groupe de données par son maximum (ou médian, etc ...)



MaxPool

Maxpool est l'autre fonction non-linéaire de choix:

- ❑ Sous-échantillonnage pour la réduction des données
- ❑ Remplace un groupe de données par son maximum (ou médian, etc ...)



SoftMax

Softmax est utilisé pour les tâches de classification car le CNN ne produit pas de valeurs binaires telles qu'une appartenance à une classe:

$$\text{softmax}(z)_i = \frac{\exp z_i}{\sum_j \exp z_j}$$

Avec z un vecteur de probabilité pour chacune des classes potentielles. $\text{softmax}(z)_i$ fait 'émerger' la classe la plus probable.

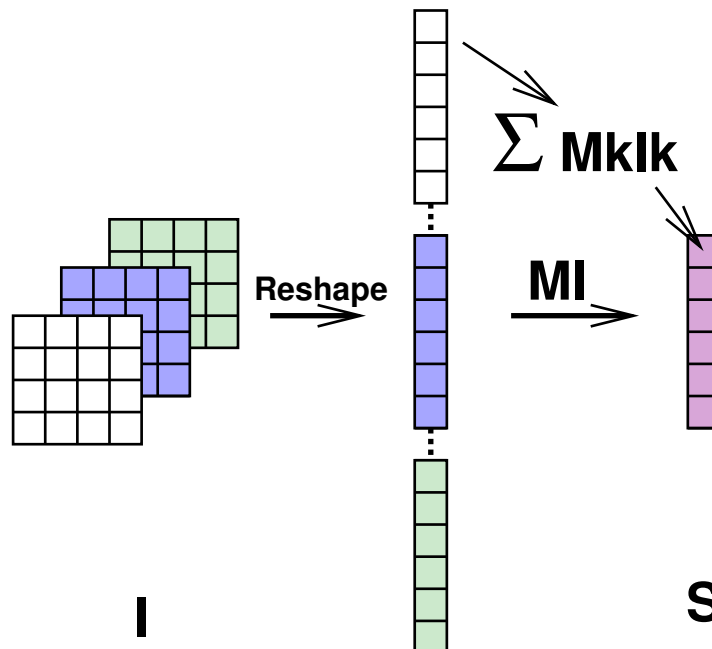
Remarque: **Softmax** provient du monde bayésien et est lié aux techniques d'optimisations statistiques basées sur le maximum de log-vraisemblance (log-likelihood).

Complètement connectée

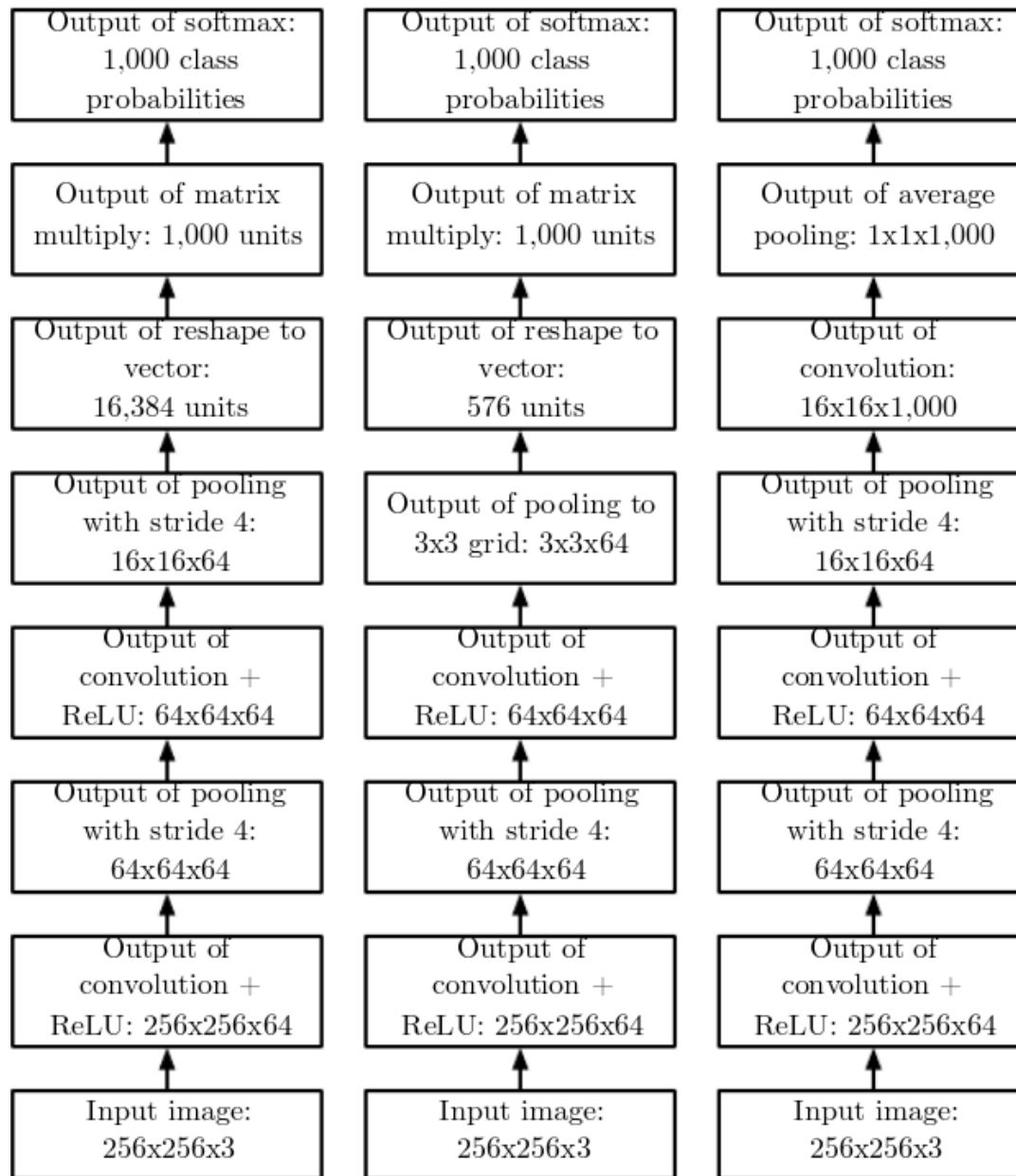
Une couches '**Fully connected**' ou **perceptron** est une combinaison linéaire de tous les éléments de la couche précédente: c'est un produit matrice - vecteur $S = M.I$, $S_i = \sum_k M_{i,k} I_k$

La feature map d'entrée (3D) est transformée en un vecteur (reshape), le vecteur de sortie peut être transformé en feature map si besoin.

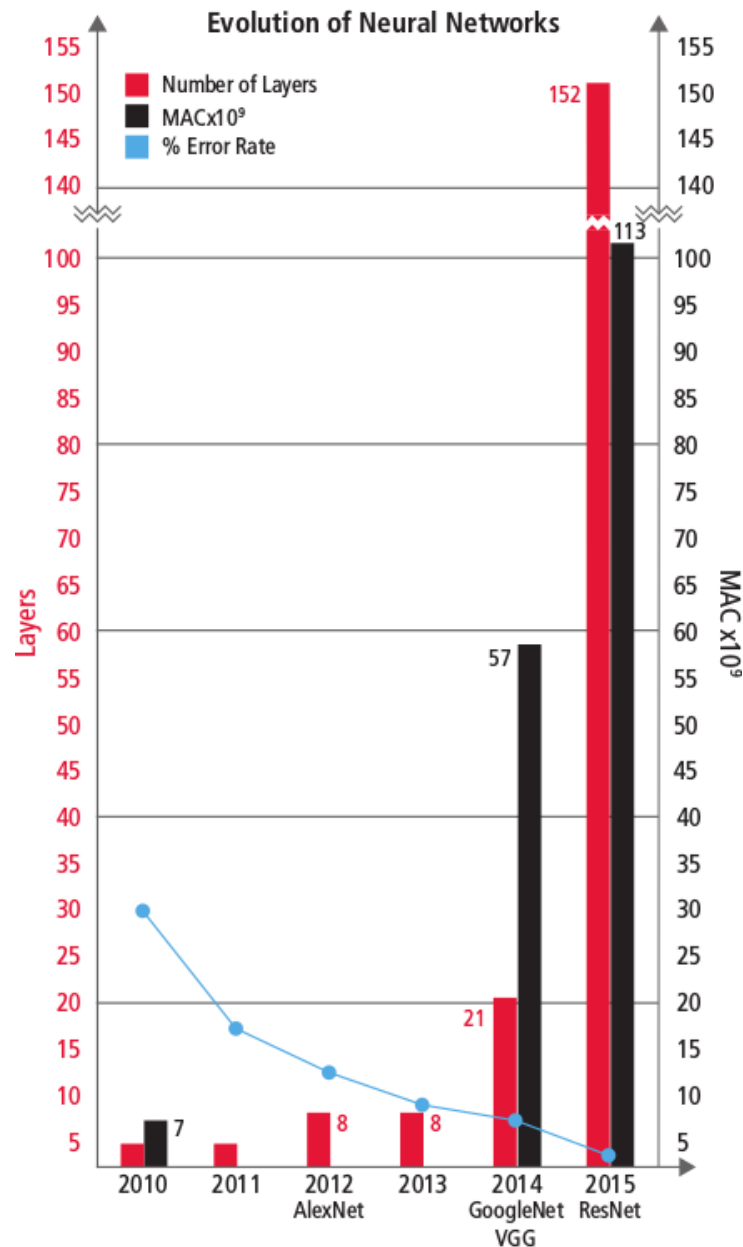
Un RNA constitué exclusivement de perceptrons est dit Multi-Layer Perceptron - MLP.



Exemples de CNN

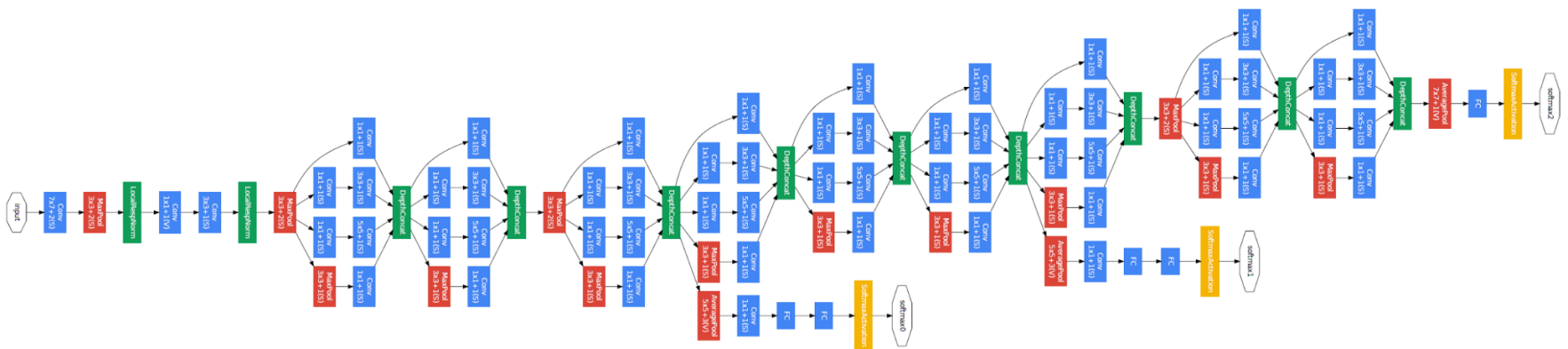


Complexité des CNN



GoogleNet - Architecture

Le réseau GoogleNet (Inception V1), fait 22 couches.
Quelques couches effectuent la **normalisation** des features map.



D'après <https://medium.com/coinmonks/paper-review-of-googlenet-inception-v1-winner-of-ilsvlc-2014-image-classification-c2b3565a64e7>

GoogleNet - Puissance de calcul

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Plan

- ❏ Virgule fixe
- ❏ Réseaux de neurones
- ✖ Architecture pour les réseaux de neurones
 - ❏ CNN sur CPU
 - ❏ Architectures spécifiques
 - ❏ CNN & GPU
- ❏ Exemple d'architectures spécifiques

Quelques constats:

- ❑ Le nombre de coefficients est très supérieur aux données d'entrée
- ❑ Les couches intermédiaires nécessitent d'être stockées en mémoire

L'accélération et la réduction de consommation peuvent être réalisés par:

- ❑ La parallélisation
- ❑ La gestion de l'arithmétique
- ❑ Les simplifications algorithmiques

Parallélisation

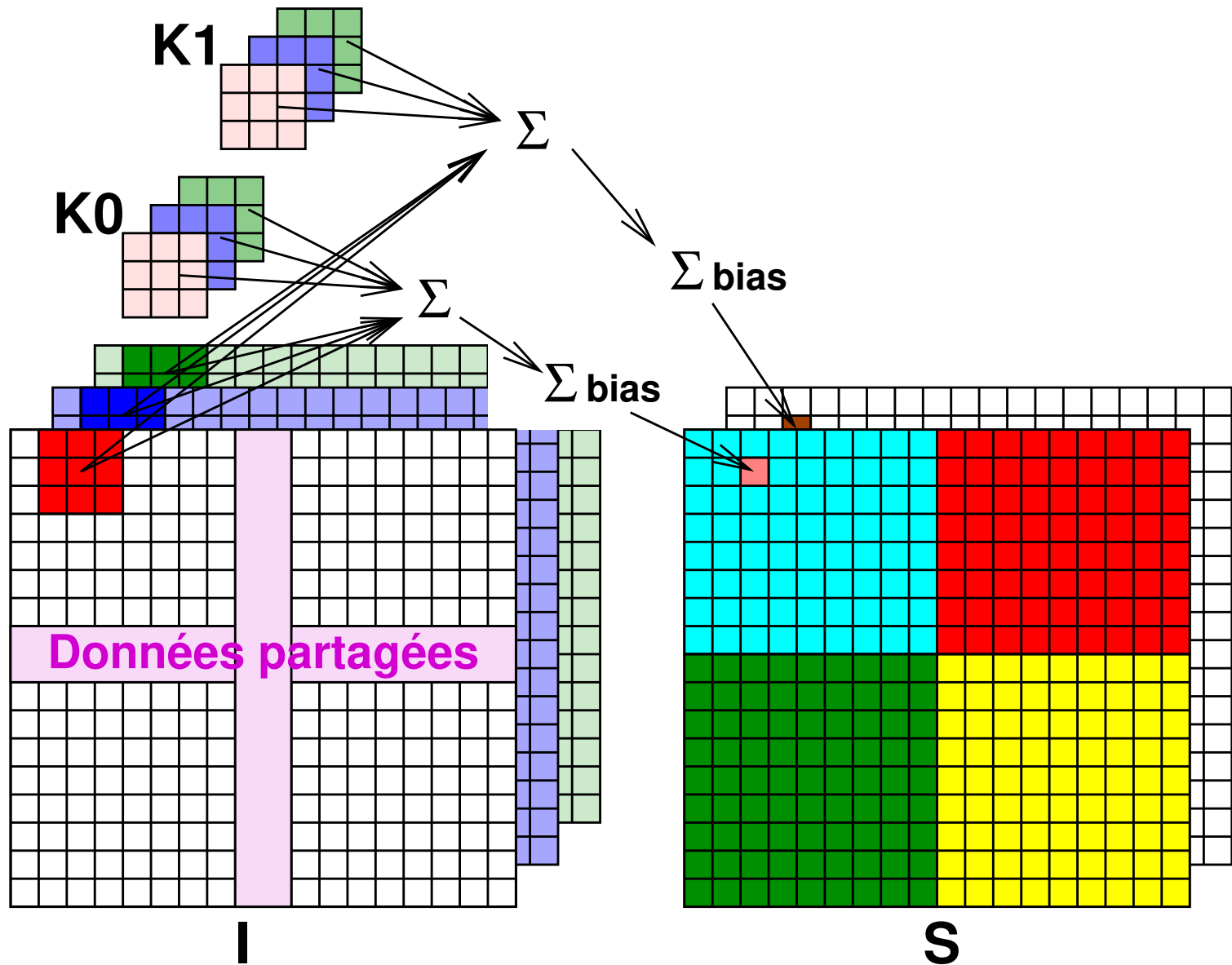
En terme d'architecture, il est théoriquement possible de paralléliser du plus bas au plus haut niveaux:

- 😊 Le calcul d'une convolution
Paralléliser les multiplications et additionner les produits
- 😊 Les calculs des éléments d'un canal de 'feature map' (les pixels)
Toutes les opérations de convolutions sont indépendantes
- 😊 Les convolutions sur chaque canal d'entrée d'une feature-map
- 😊 Les canaux pour produire une feature-map
- 😊 Calculer des régions de feature map différentes

Mais des surcoûts réduisent l'efficacité:

- 🍄 Les Kernel (3x3, 5x5, . . .) ne sont pas alignés sur une puissance de 2!
- 🍄 Les couches d'entrée sont de grandes tailles, avec 'peu' de canaux
- 🍄 Les couches profondes sont des petite tailles, avec de très nombreux canaux

Parallélisation "haut niveau"

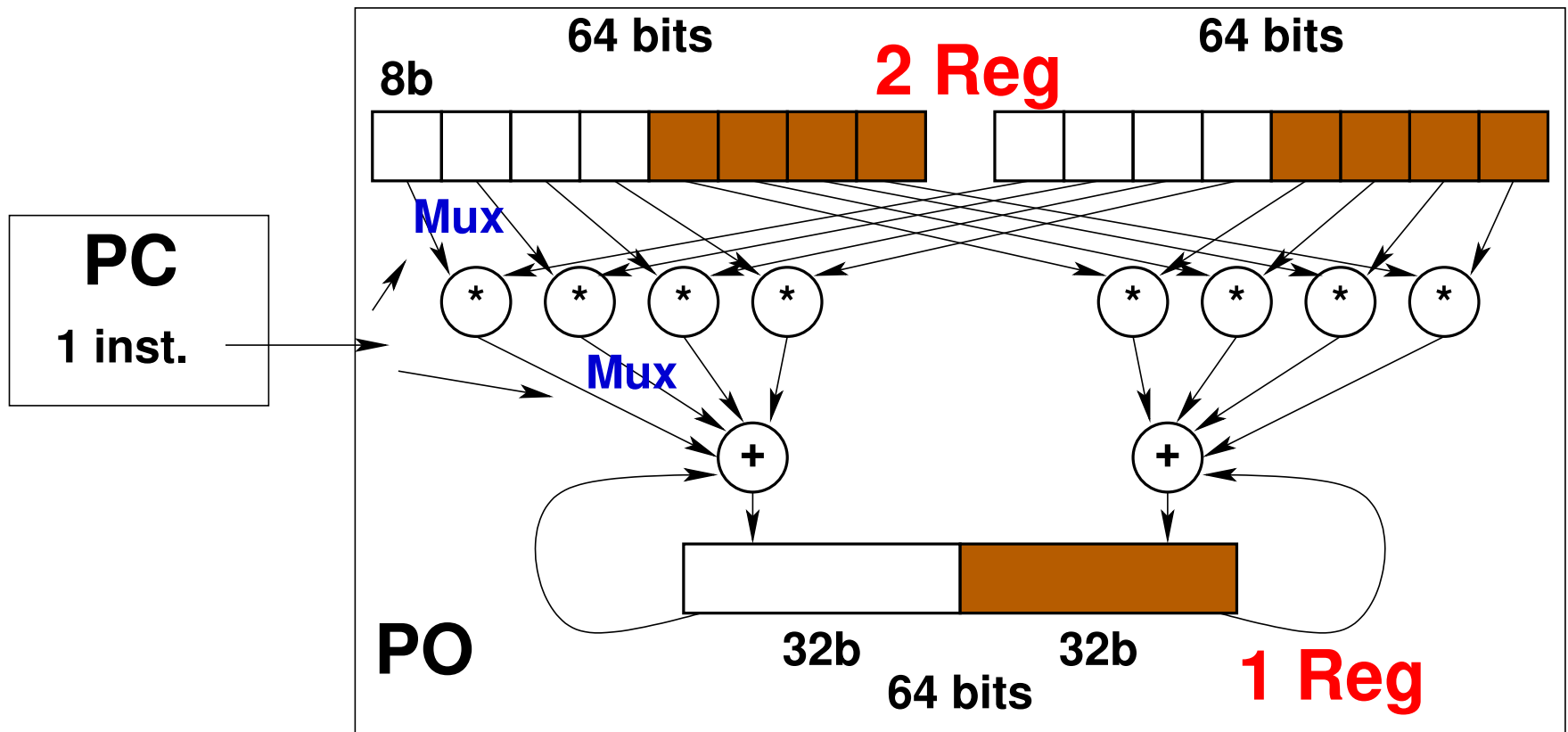


- ❑ Virgule fixe
- ❑ Réseaux de neurones
- ✖ Architecture pour les réseaux de neurones
 - ✓ CNN sur CPU
 - ❑ Architectures spécifiques
 - ❑ CNN & GPU
- ❑ Exemple d'architectures spécifiques

Vectorisation sur CPU

Les unités vectorielles permettent d'effectuer des calculs parallèles sur des sous-mots des registres.

La vectorisation peut être combinée à du pipeline et du n-issue (VLIW).



Vectorisation & CNN

Les vecteurs peuvent être des entier/flottants selon la taille des opérandes.

😊 Disponible sur de nombreux processeurs (NEON, SSE, ...)

😊 Puissance de calcul accrue

🍄 Gestion de la précision

🍄 Alignement/désalignement des données

🍄 Gestion des registres sources/destination

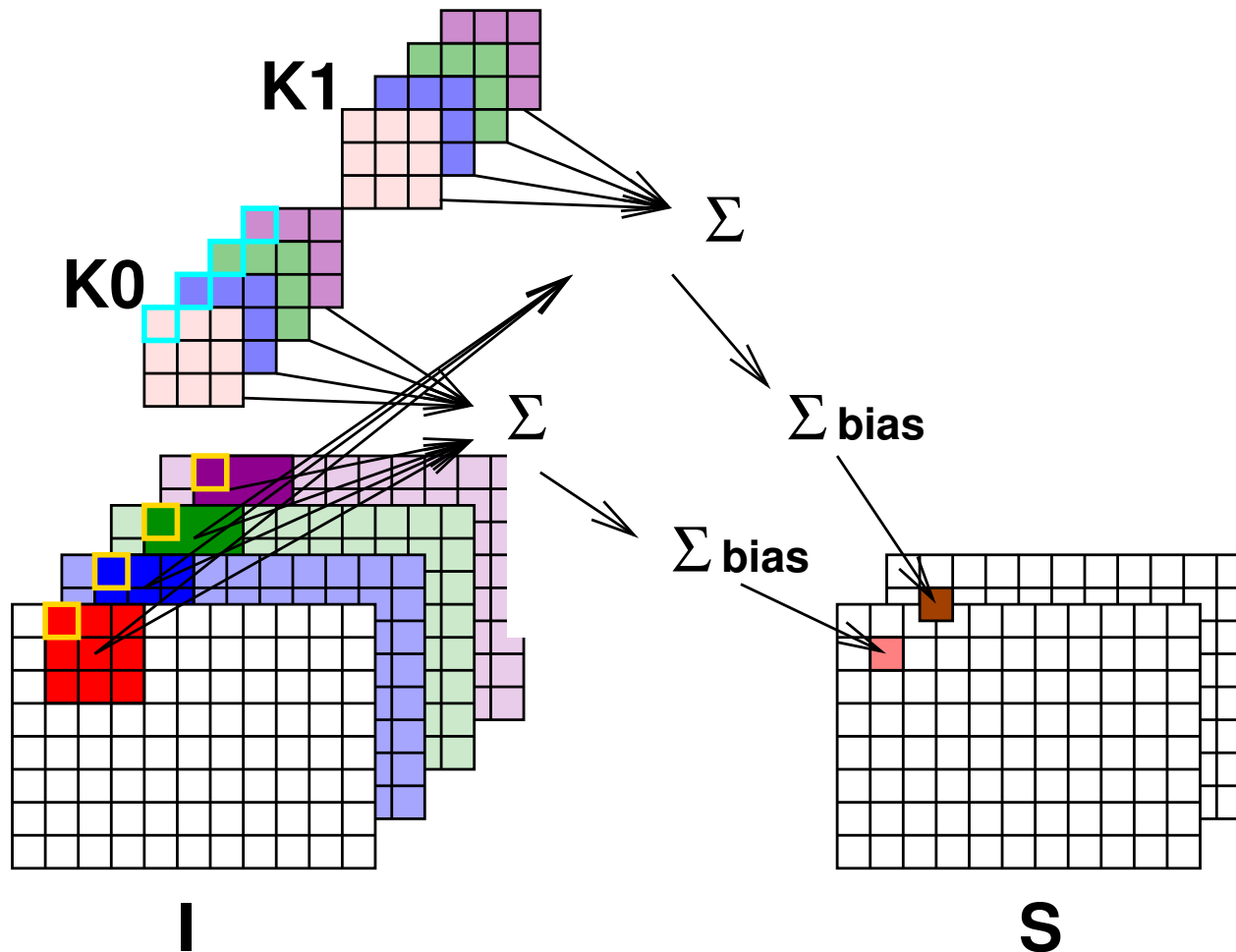
Vectorisation et alignement

Quelques contraintes:

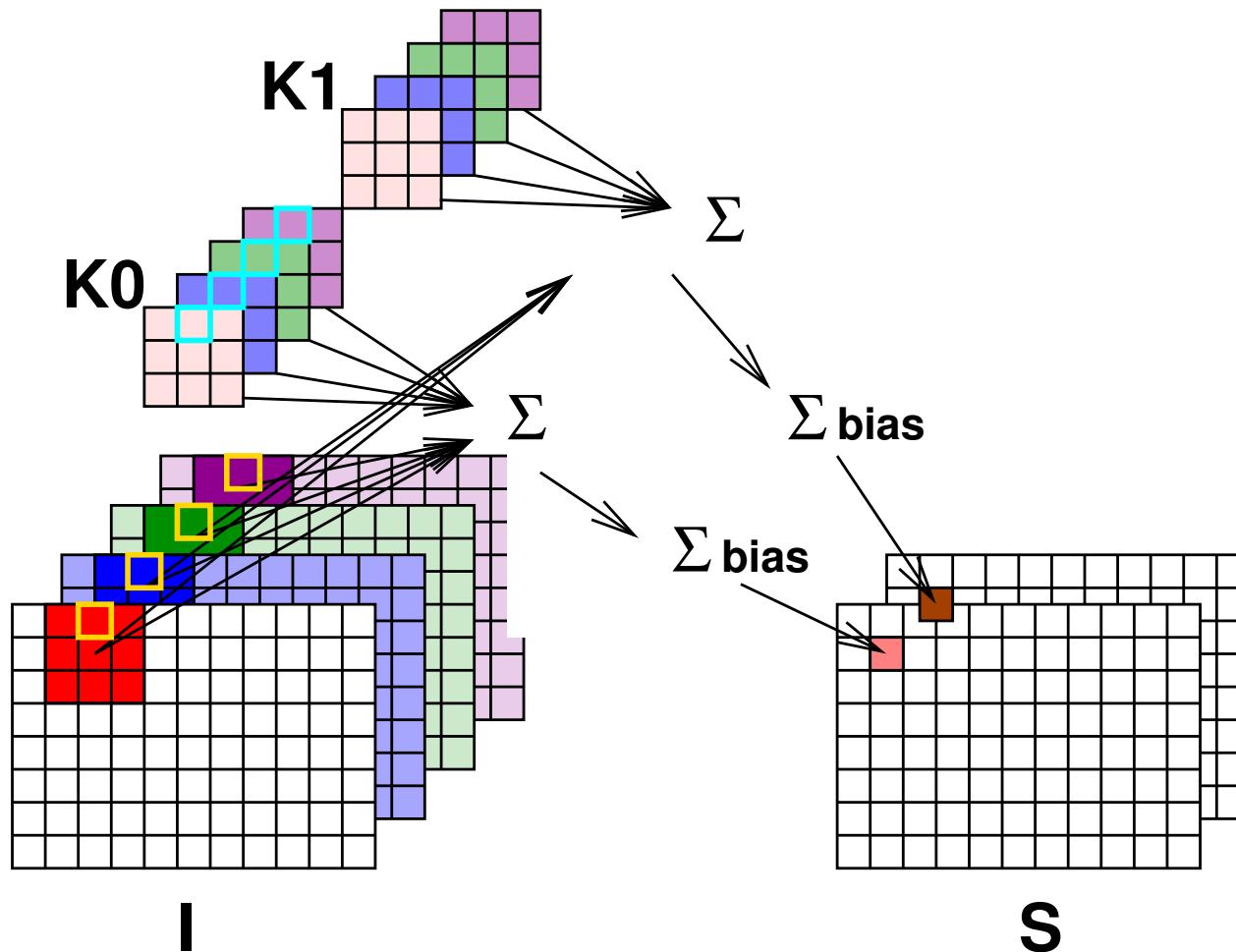
- ❑ Un vecteur est constitué de mots mémoire contigus, de taille en puissance de 2
- ❑ Les filtres de convolutions sont de taille $3 * 3$ ou $5 * 5$
- ❑ L'application d'un filtre sur une feature map impose d'accéder à une fenêtre de la taille du filtre
- ❑ Les feature map sont rangées en adresse contigües selon la largeur ou bien selon le canal
- ❑ Les features map peuvent avoir une taille (nombre de canal) non multiple de la taille du vecteur

➤ La vectorisation est associée à un rangement spécifique en mémoire et à une ré-organisation des calculs.

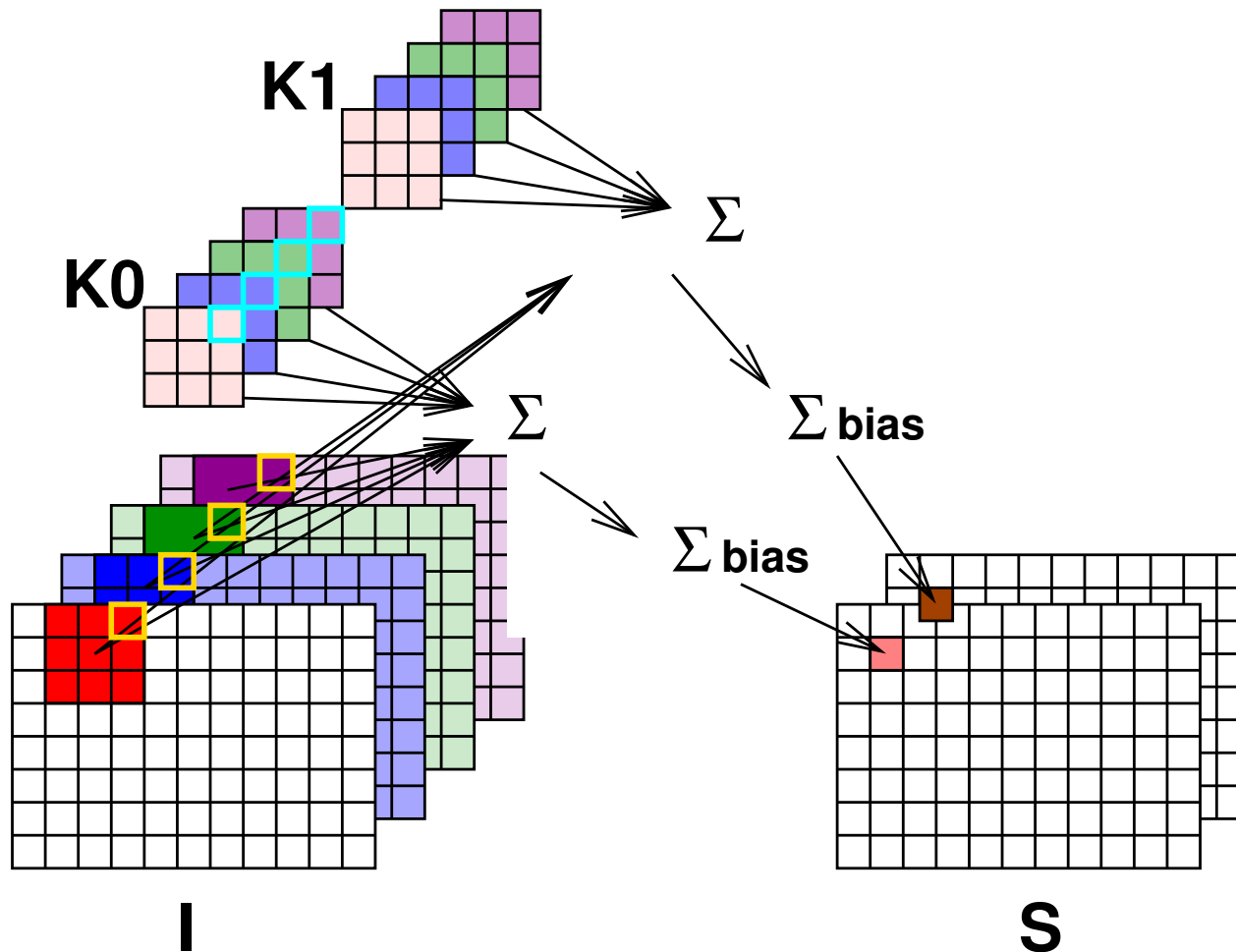
Vectorisation selon les canaux



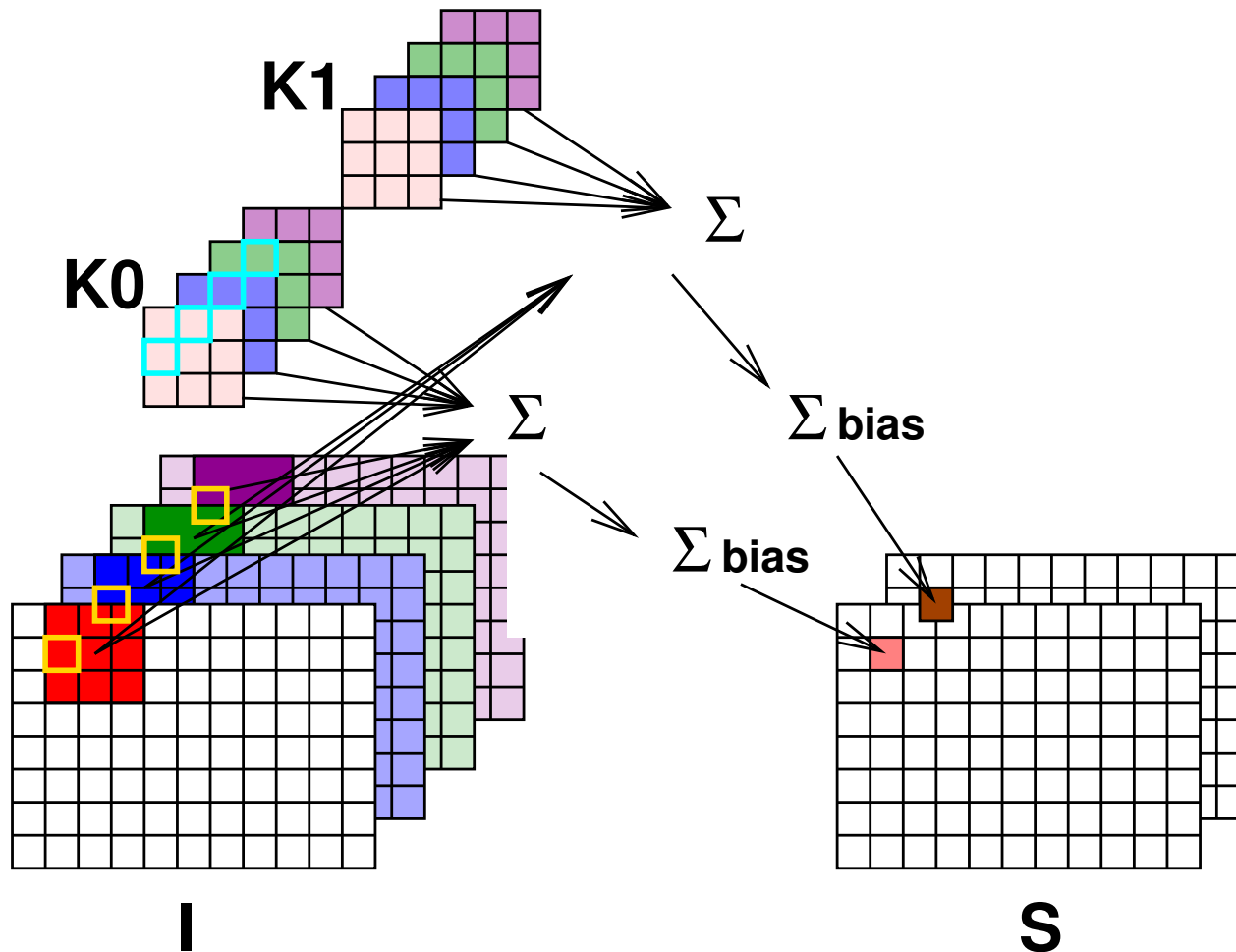
Vectorisation selon les canaux



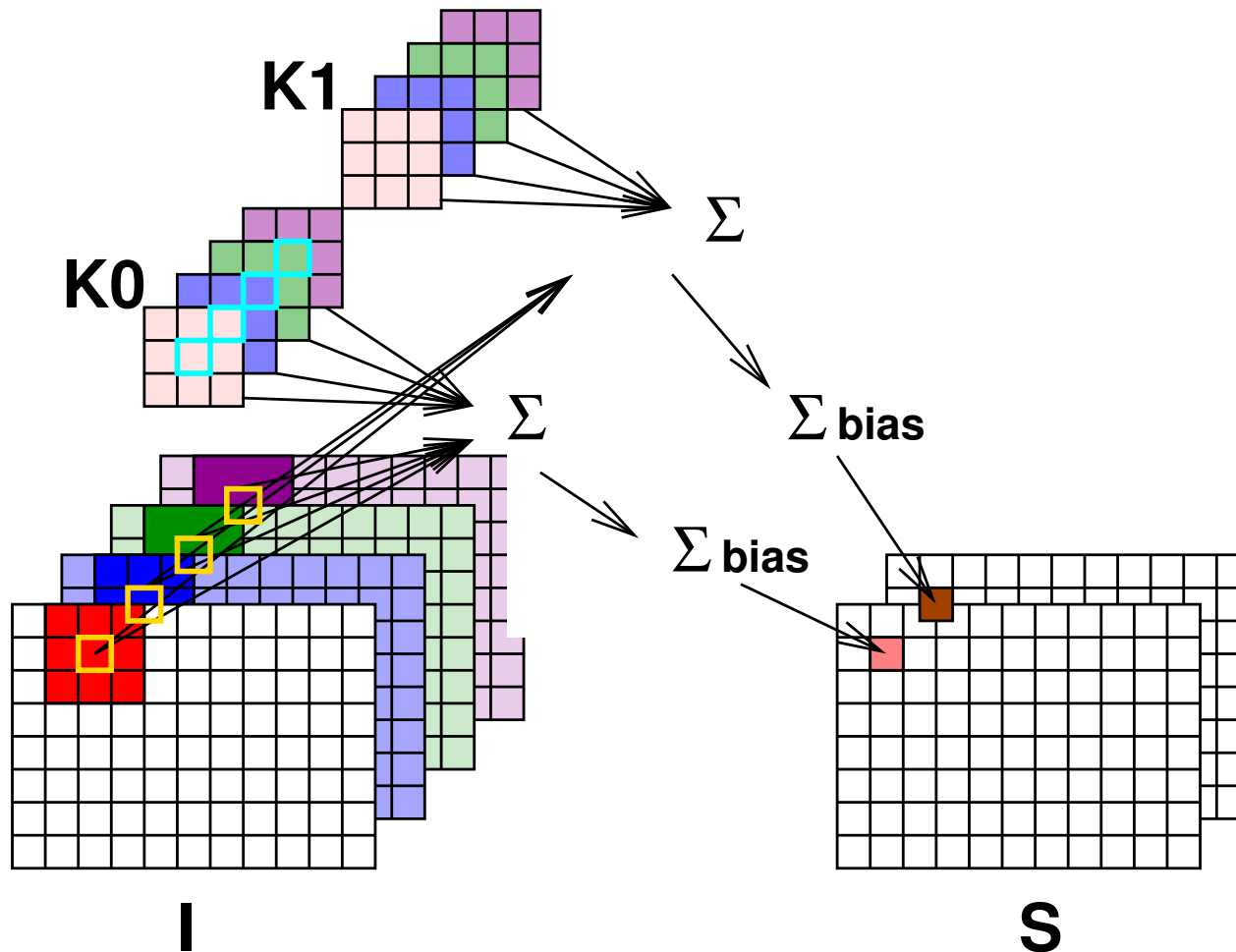
Vectorisation selon les canaux



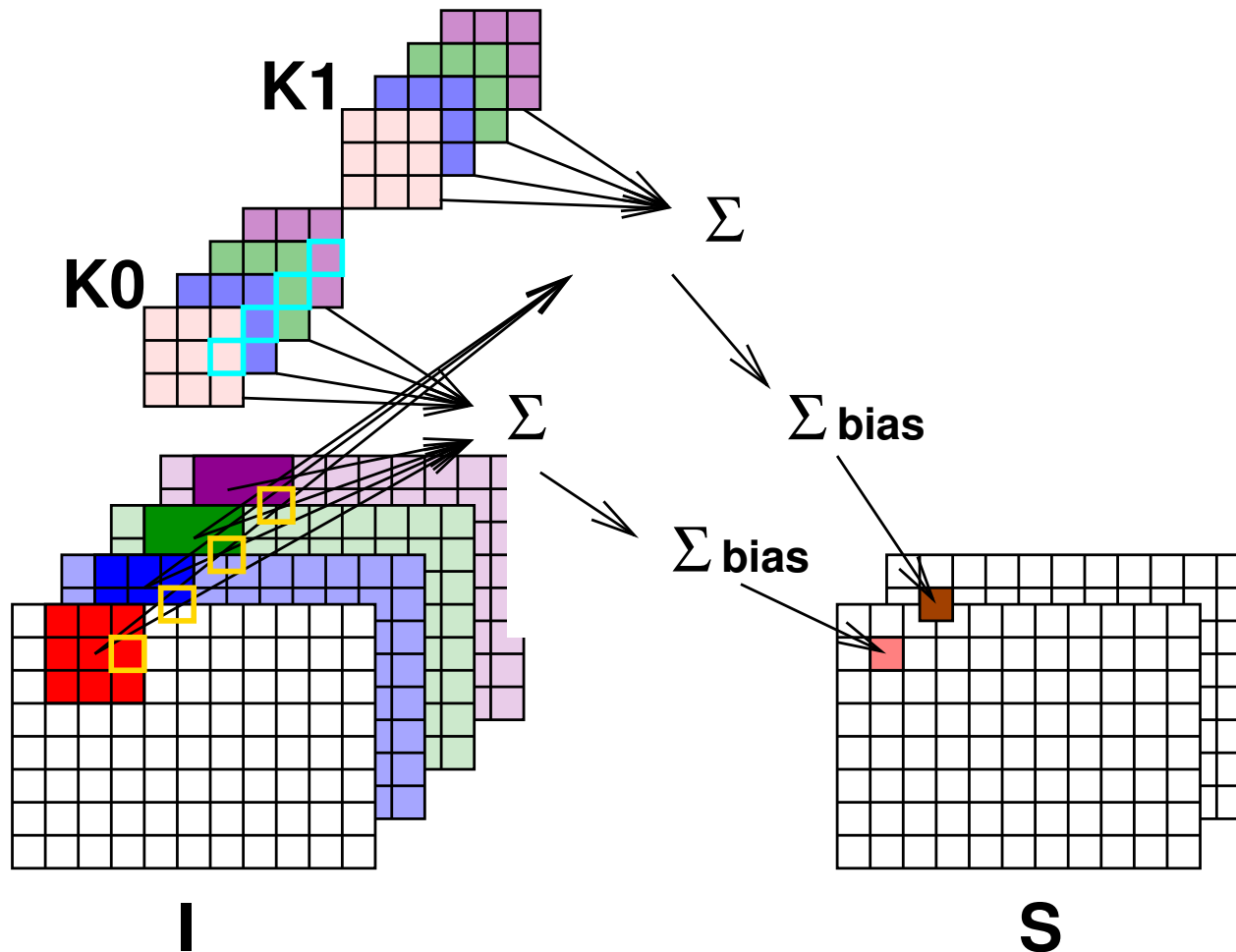
Vectorisation selon les canaux



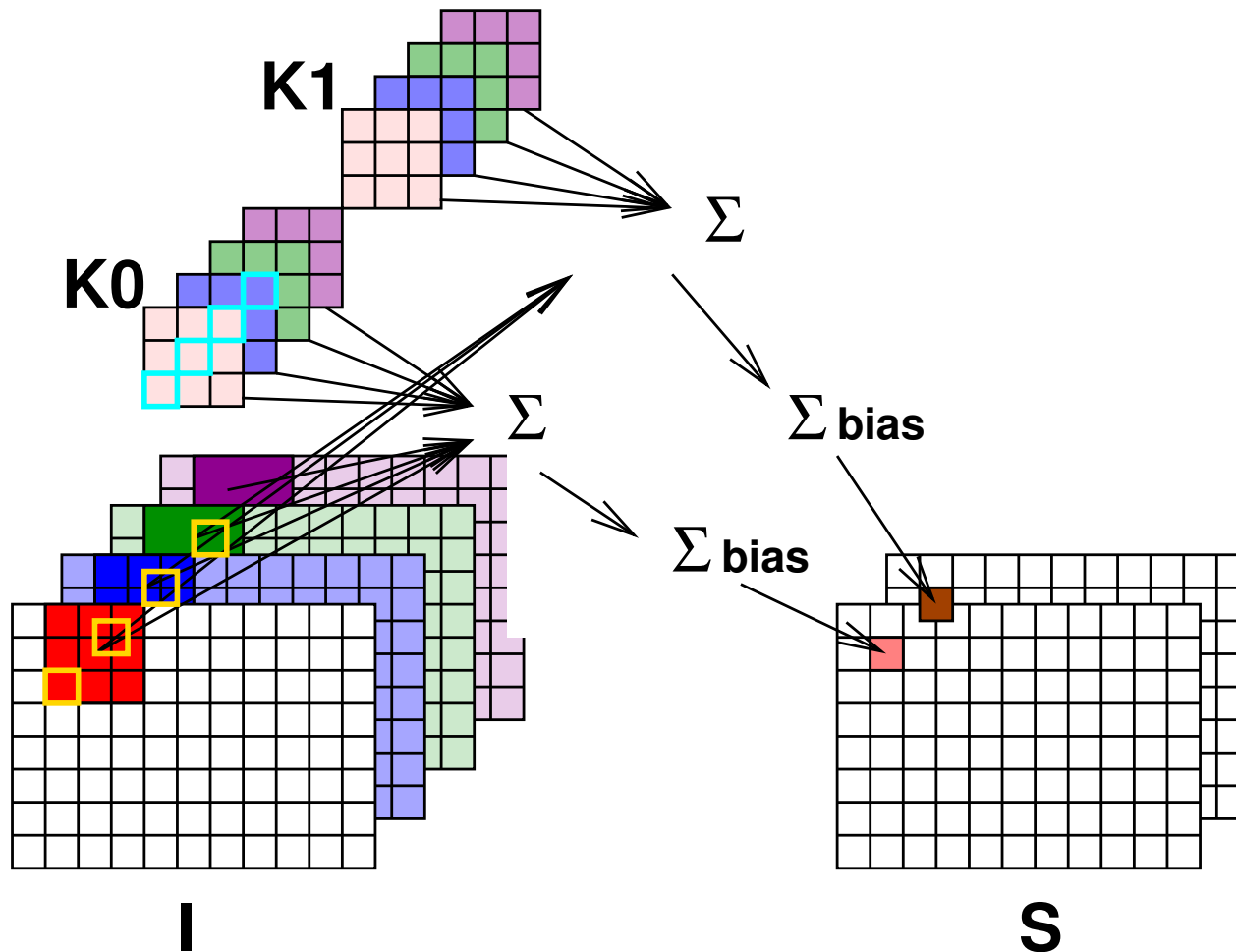
Vectorisation selon les canaux



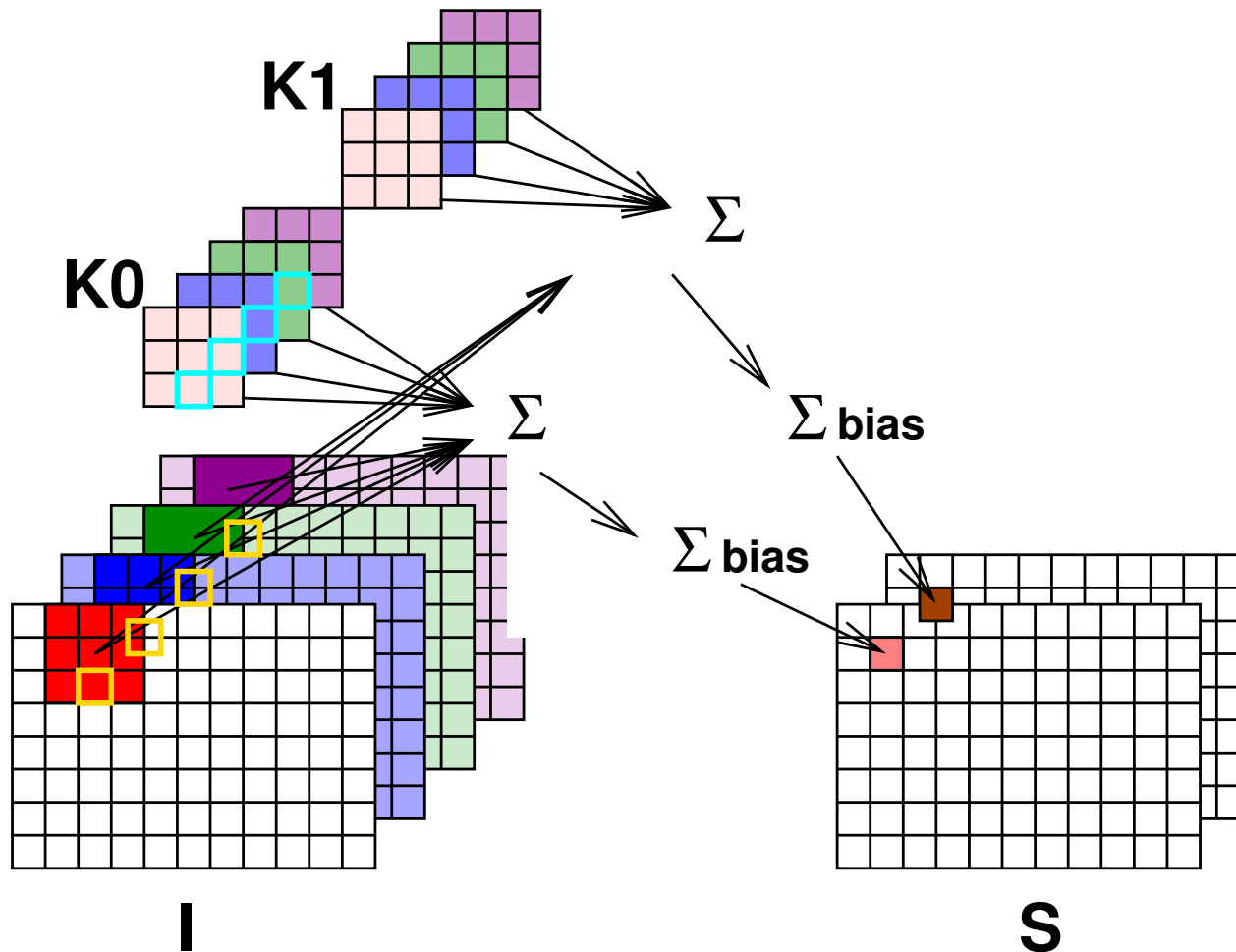
Vectorisation selon les canaux



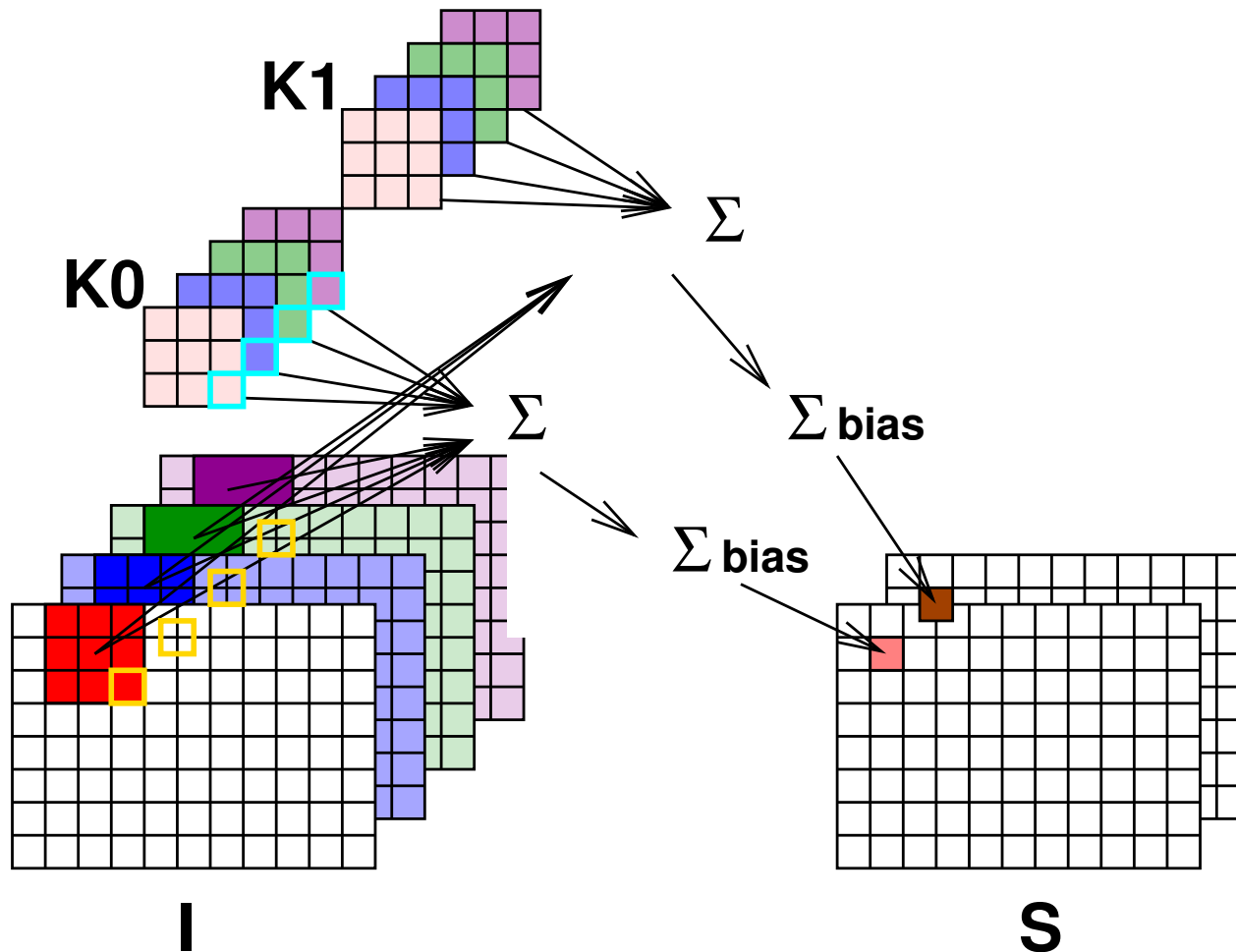
Vectorisation selon les canaux



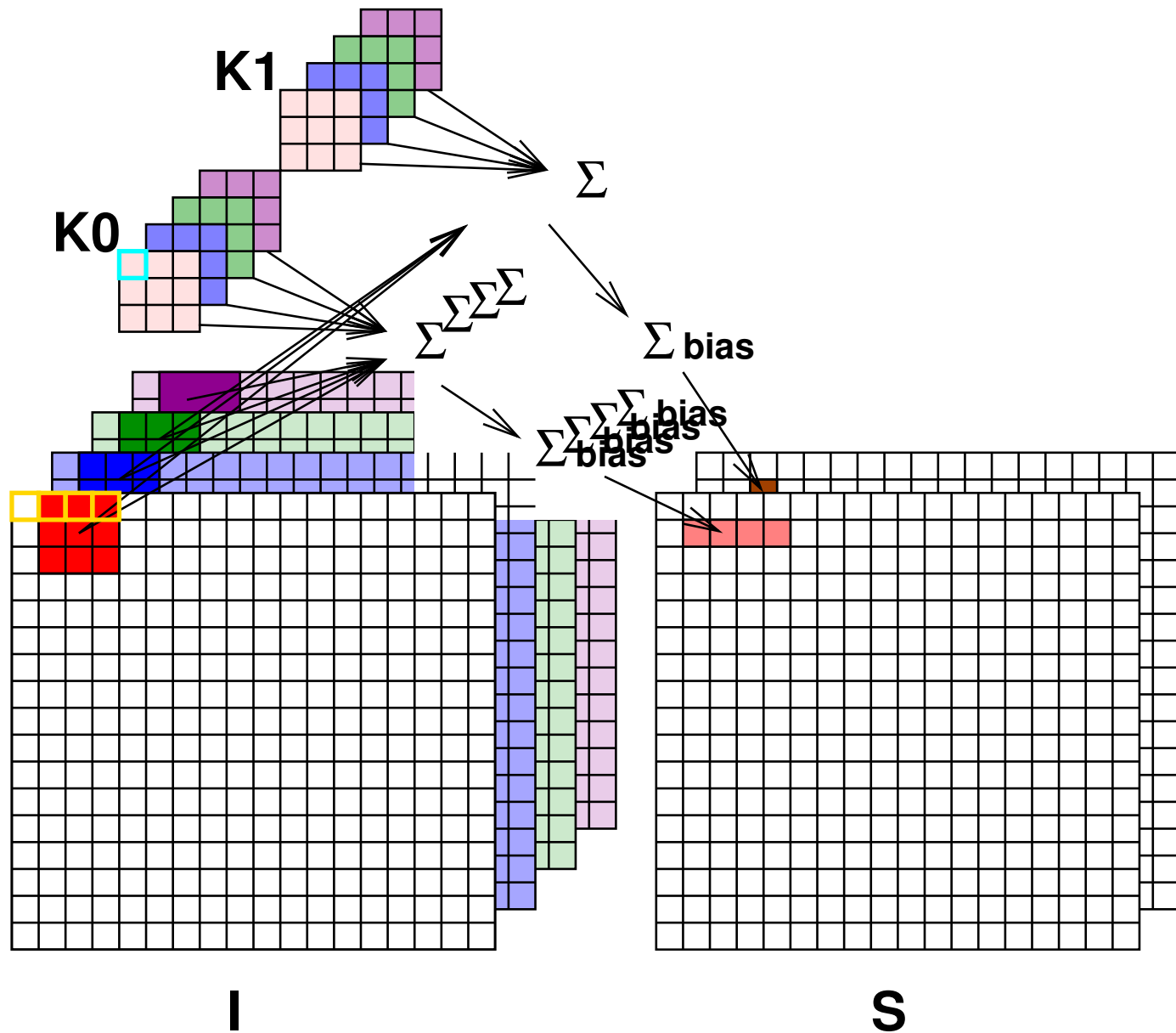
Vectorisation selon les canaux



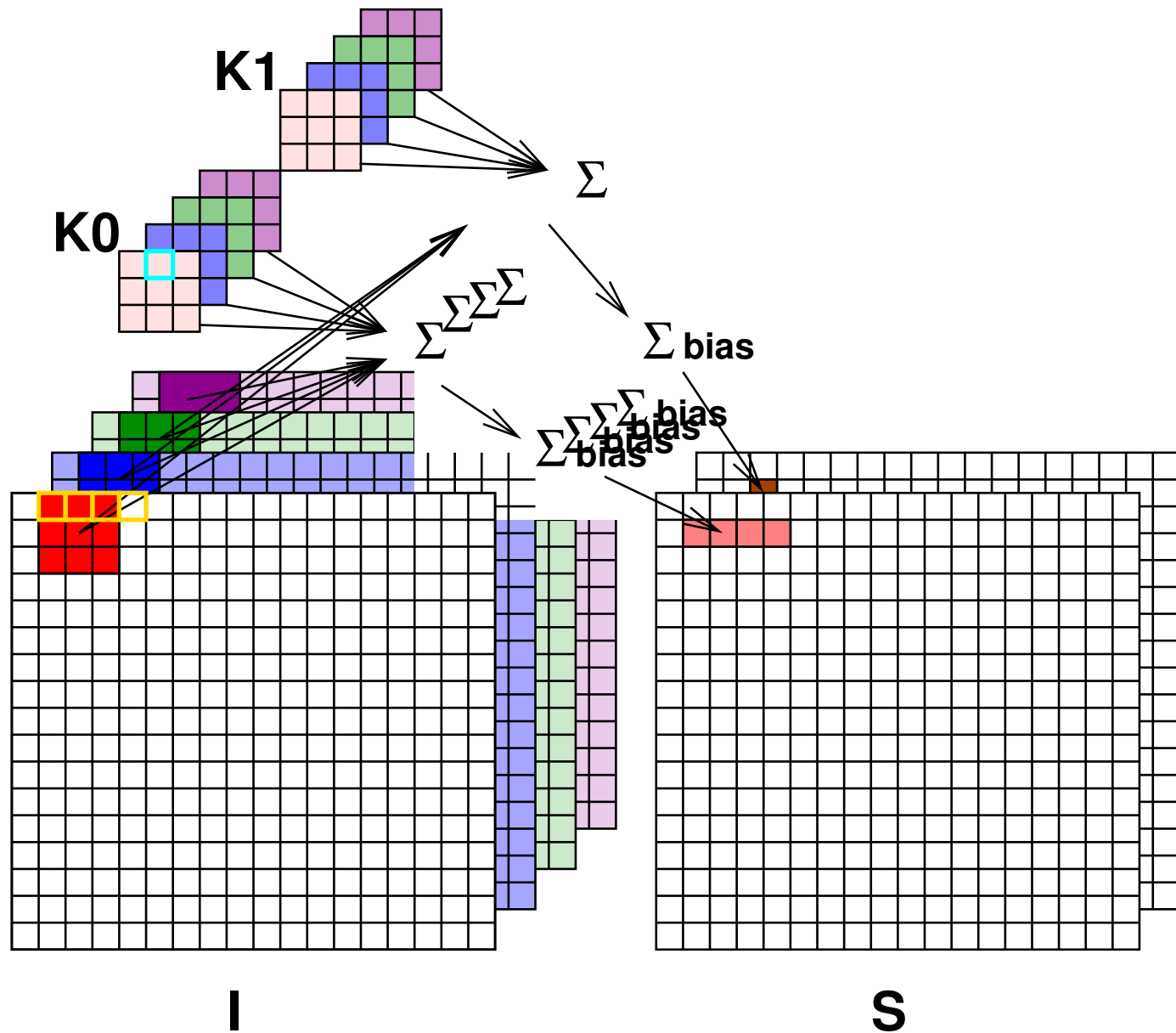
Vectorisation selon les canaux



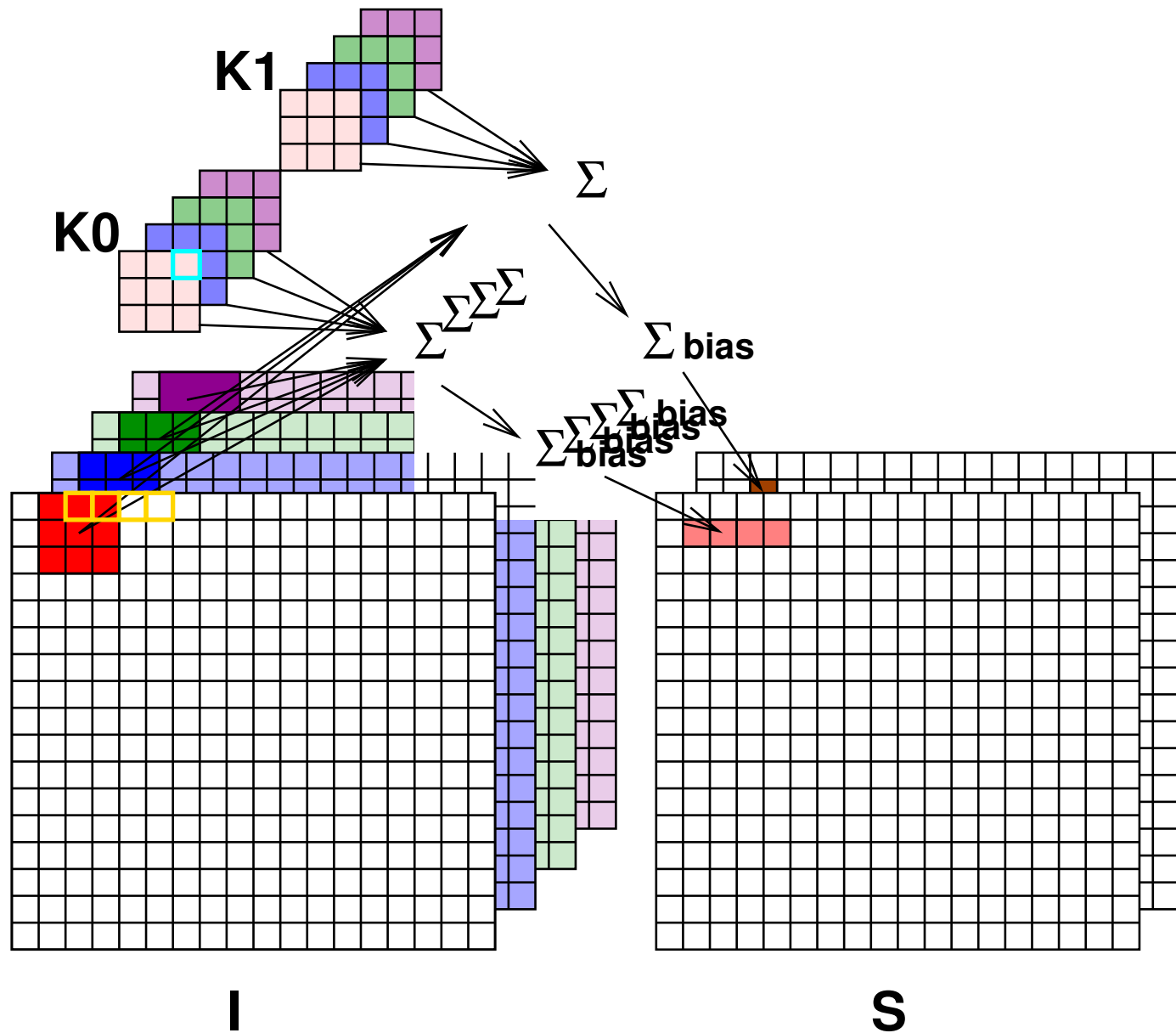
Vectorisation selon les éléments



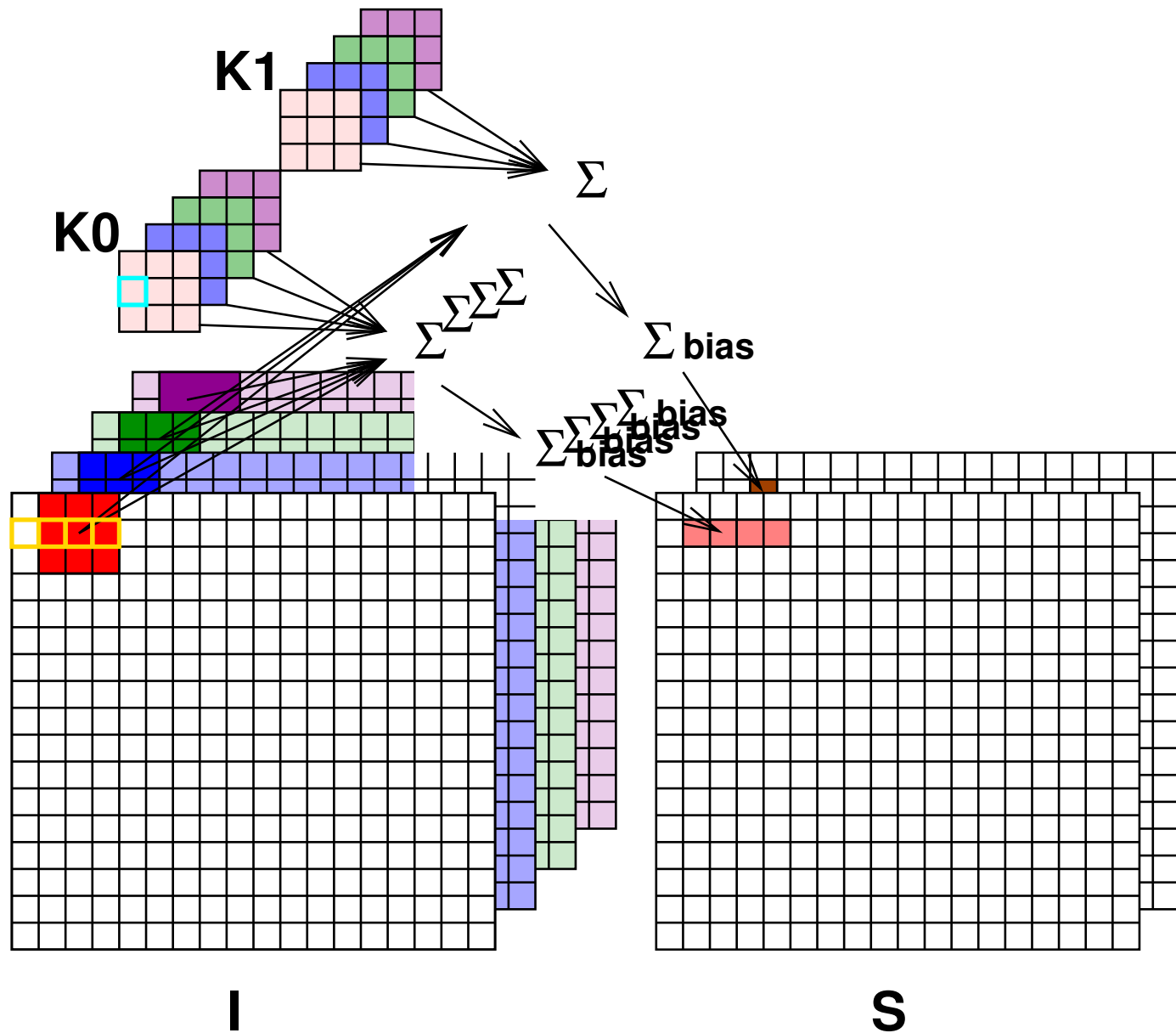
Vectorisation selon les éléments



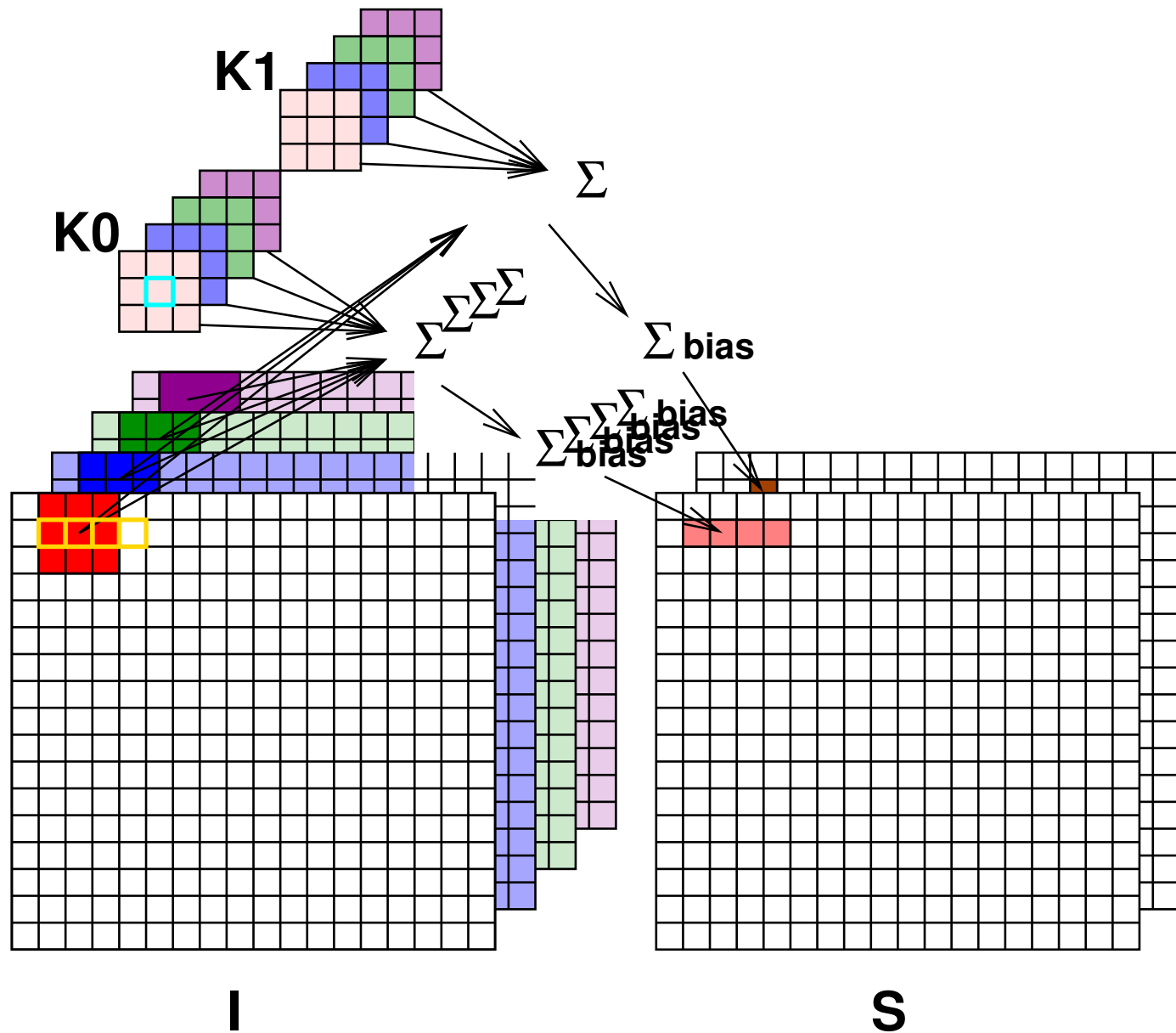
Vectorisation selon les éléments



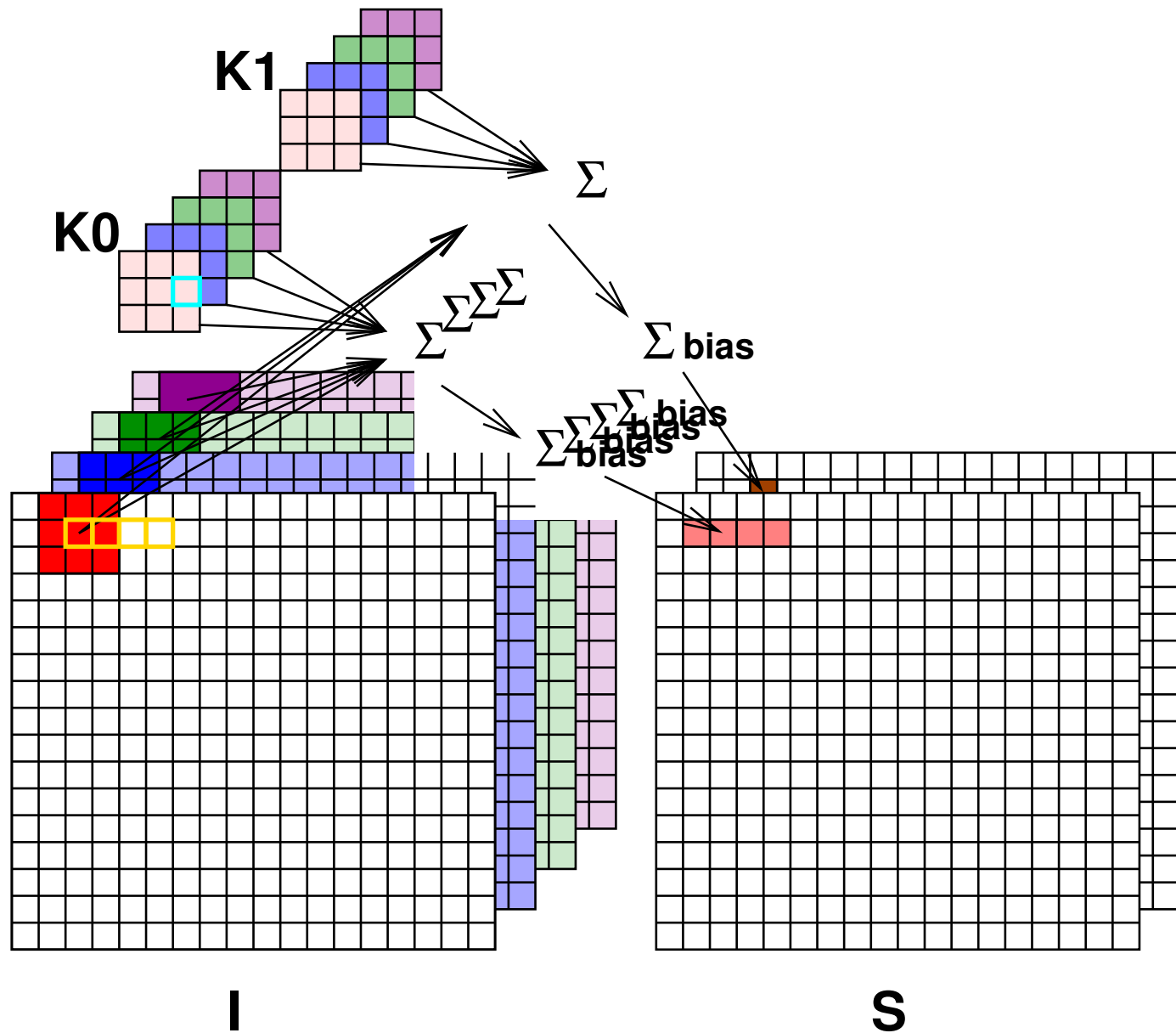
Vectorisation selon les éléments



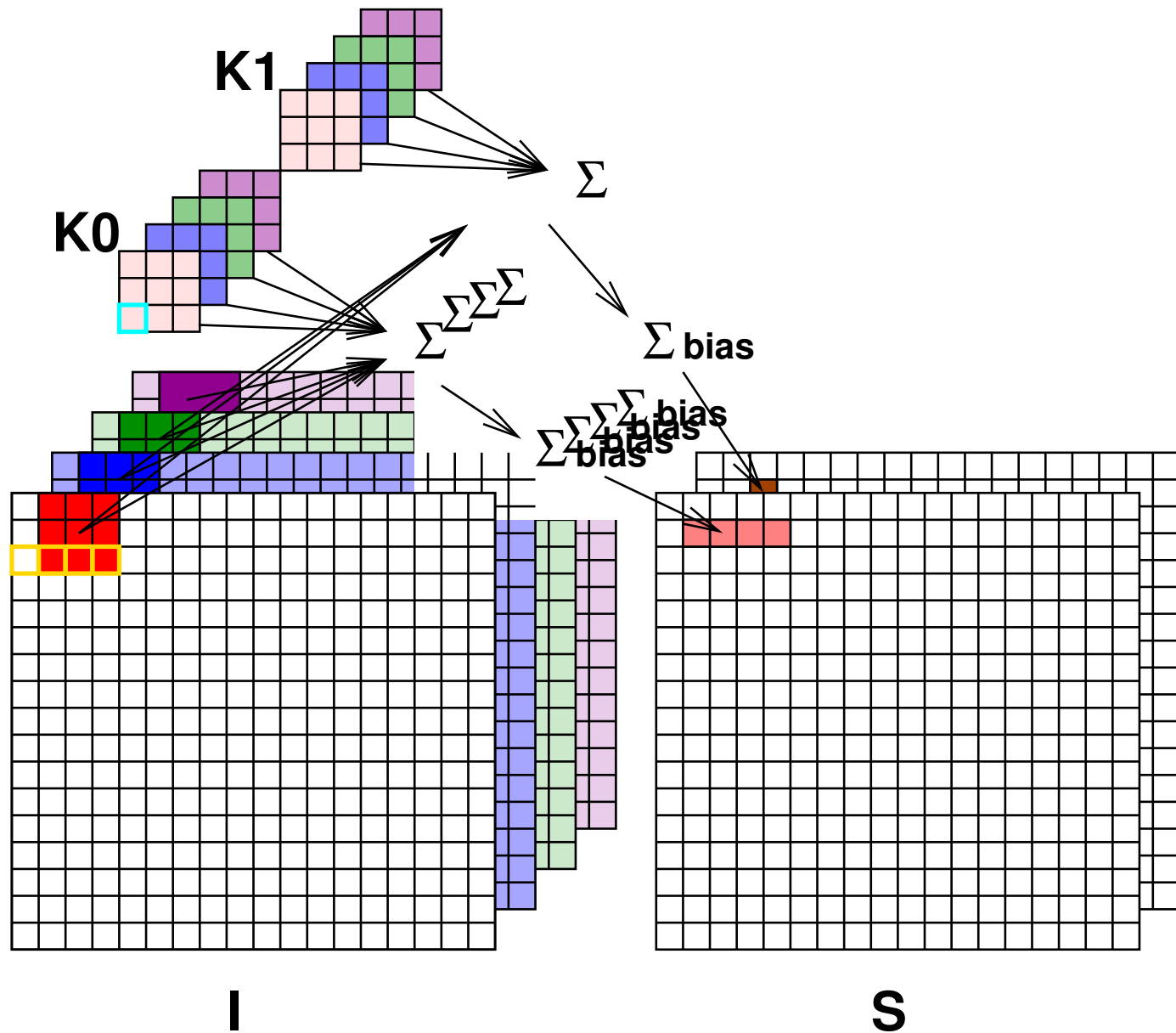
Vectorisation selon les éléments



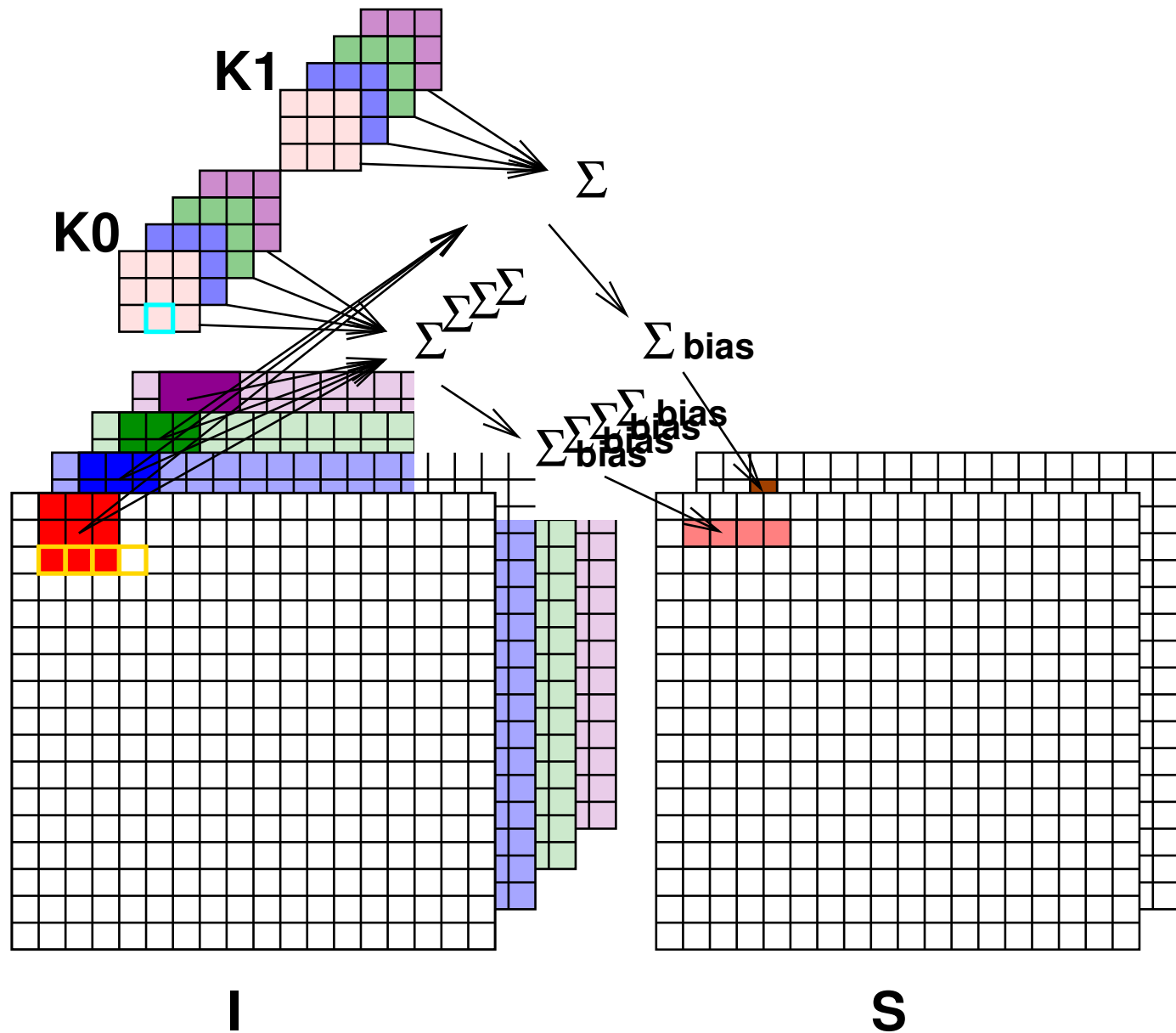
Vectorisation selon les éléments



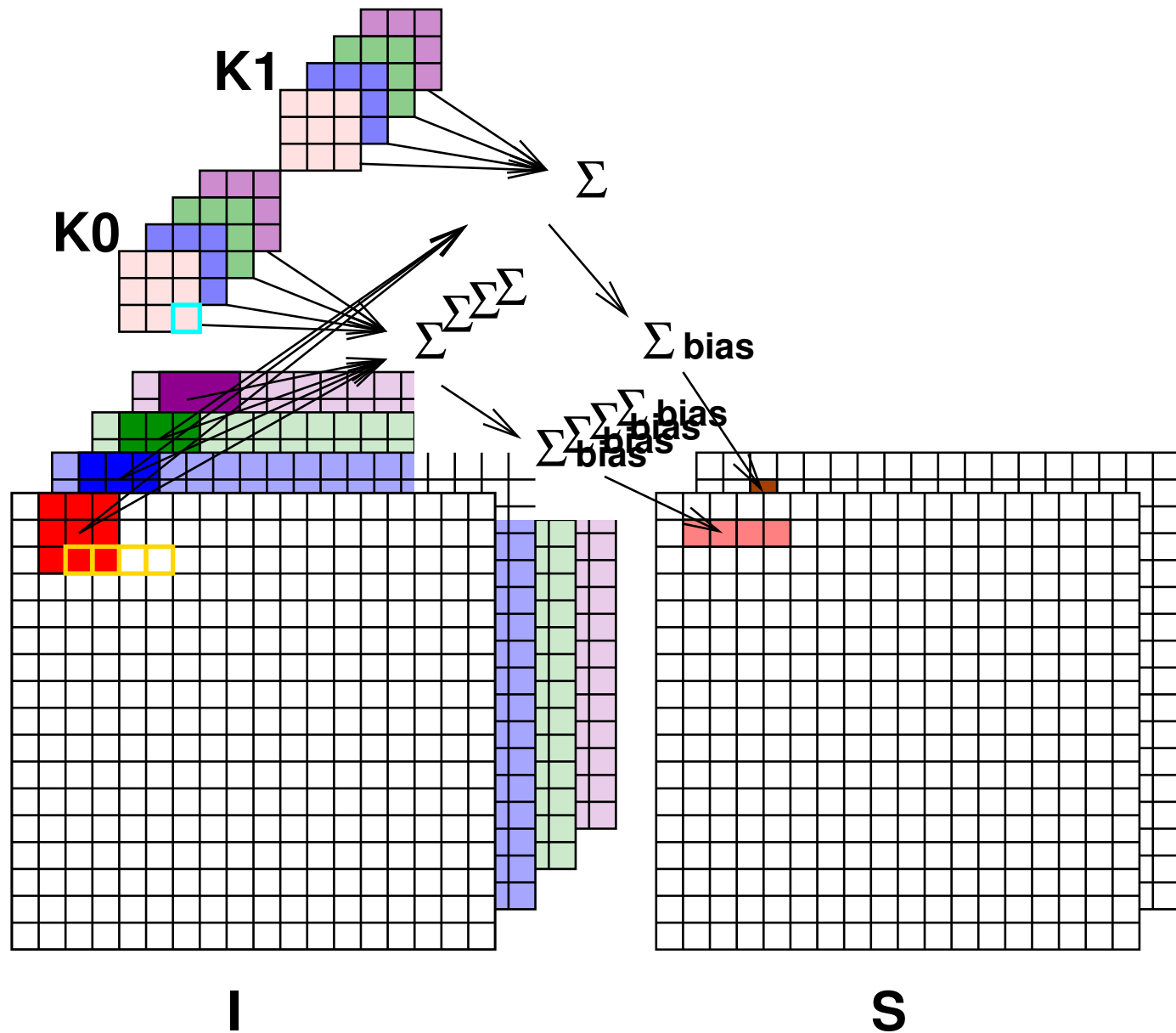
Vectorisation selon les éléments



Vectorisation selon les éléments



Vectorisation selon les éléments



- ❑ Virgule fixe
- ❑ Réseaux de neurones
- ✖ Architecture pour les réseaux de neurones
 - ❑ CNN sur CPU
 - ✓ Architectures spécifiques
 - ❑ CNN & GPU
- ❑ Exemple d'architectures spécifiques

Opportunités des architectures spécialisées

- 😊 Tous les niveaux de parallélisme sont possibles
- 😊 Les FPGA/ASIC autorisent une meilleure gestion des (dés)-alignement
- 😊 Il existe de très nombreuses variantes et compromis surface/performance
- 😊 Il est possible de construire des dispositifs pour paralléliser au plus bas niveau, en utilisant des mémoires SRAM et registres.
- 😊 Les 'feature-map' et coefficients peuvent transiter en **pipeline** dans l'architecture mais leur taille impose très souvent la mise en place d'une **hiérarchie mémoire**.
- 😊 Les sommes multiples sont réalisées à l'aide d'arbres d'additionneurs

Architectures de calcul

Comment réaliser matériellement le calcul de convolution?

Quelles sont les architectures qui permettent différents compromis temps de calcul/surface pour implémenter le calcul:

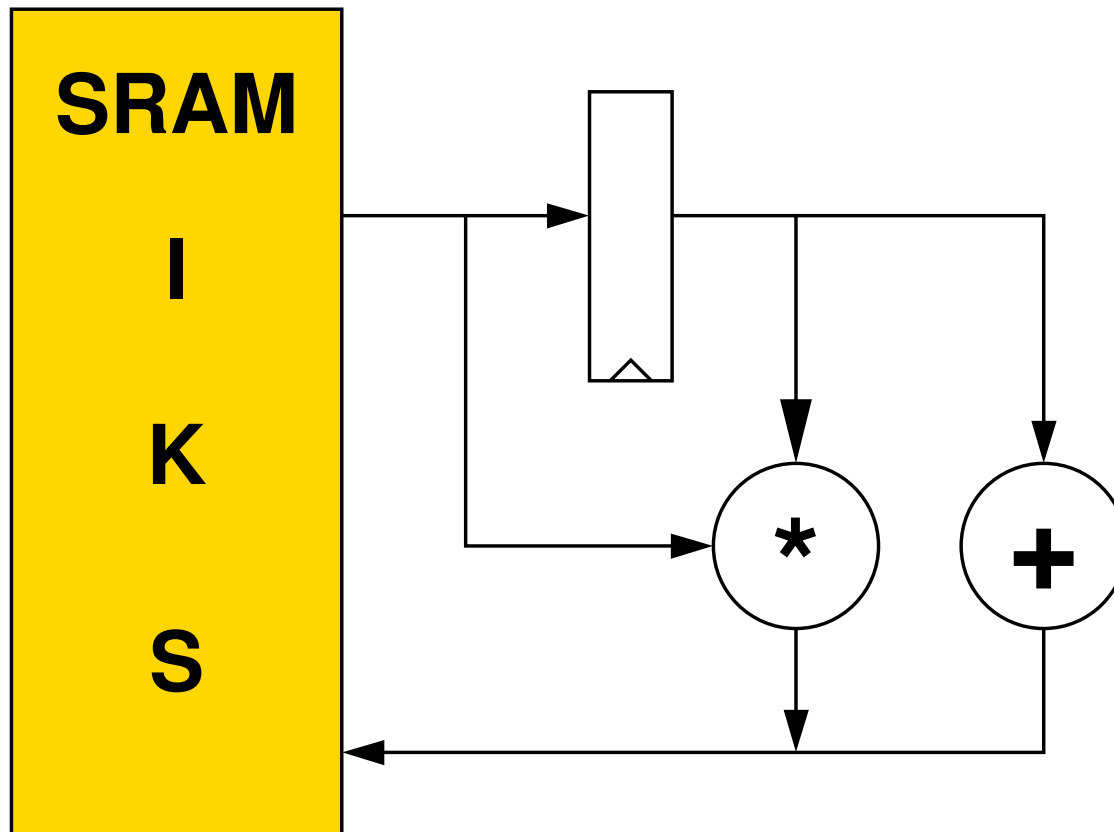
$$S_{c,i,j} = \sum_{l,m,n} I_{l,i+m,j+n} K_{c,l,m,n} + \text{bias}_c$$

Dans un premier temps, on suppose que les données et coefficients sont dans des mémoires **SRAM**.

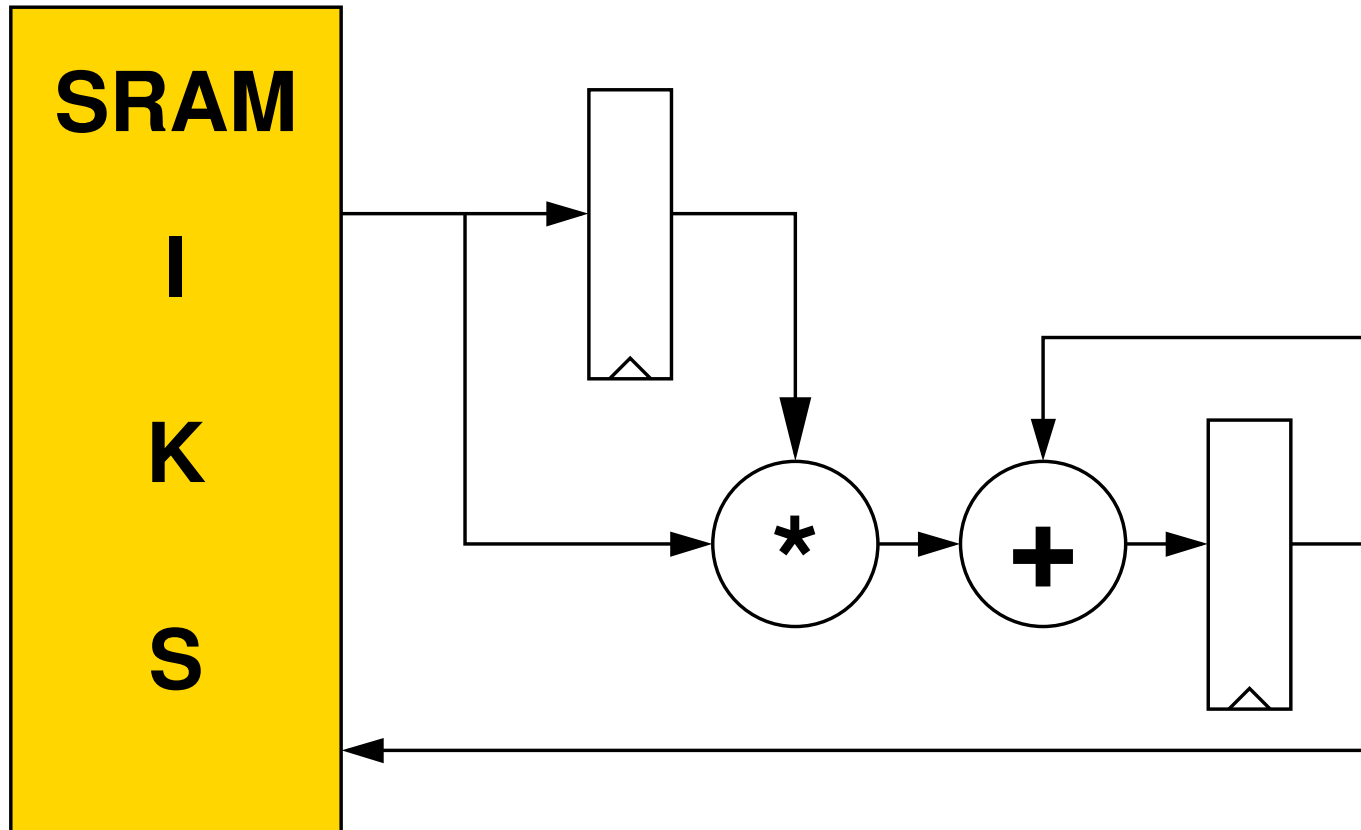
On commencera par l'architecture la plus simple (et la moins efficace), puis on la rendra de plus en plus complexe.

L'hypothèse des données en SRAM sera levée plus tard, et on suppose qu'il existe un moyen de les transférer vers des DDR-SDRAM.

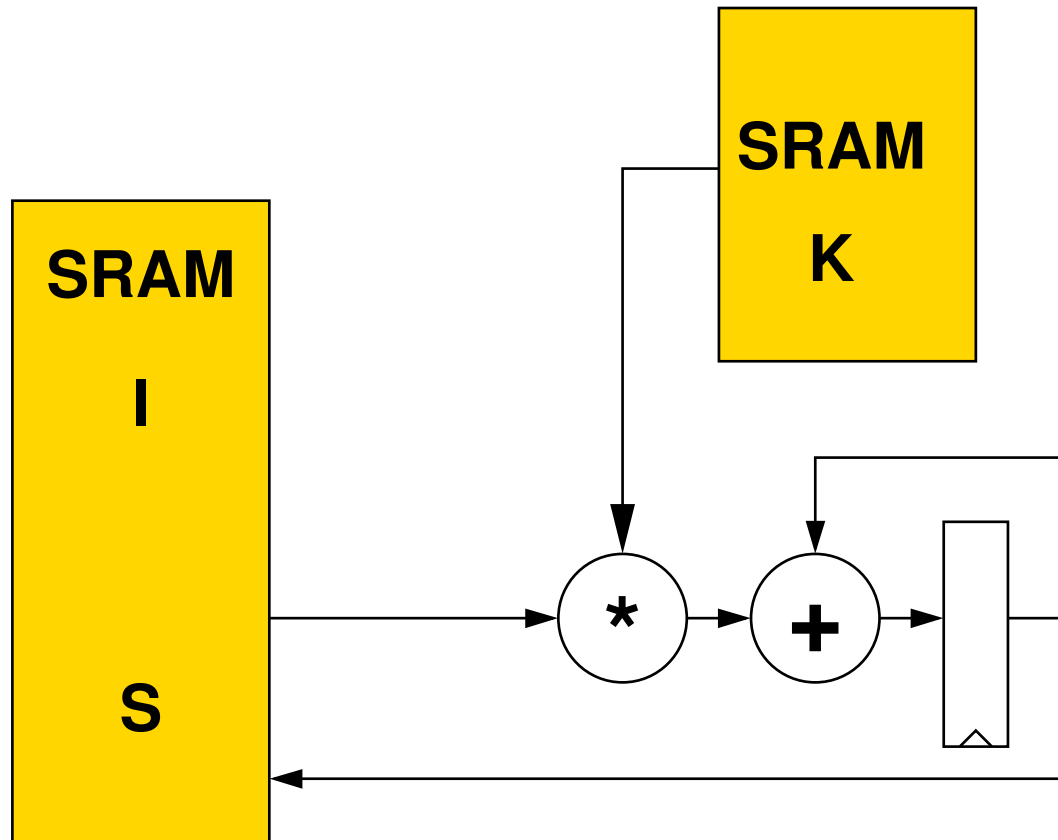
Mémoire unique



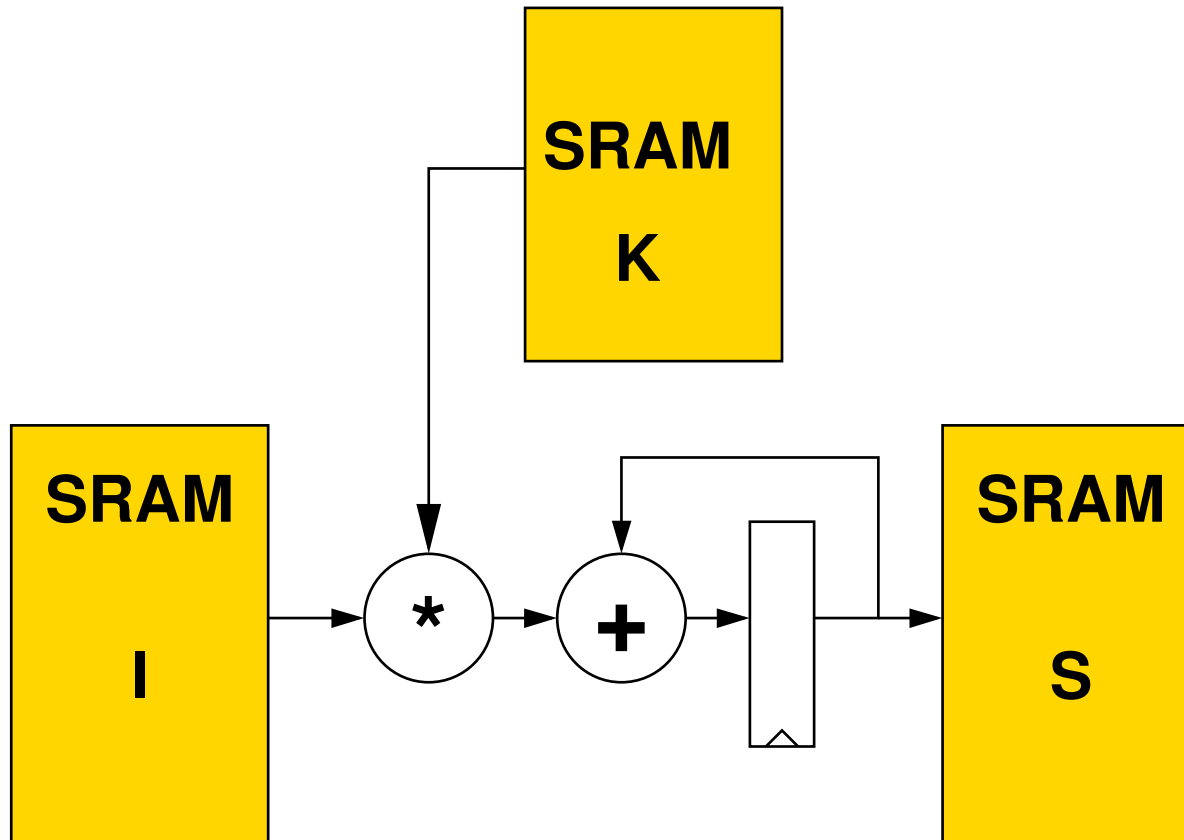
Une mémoire et un registre



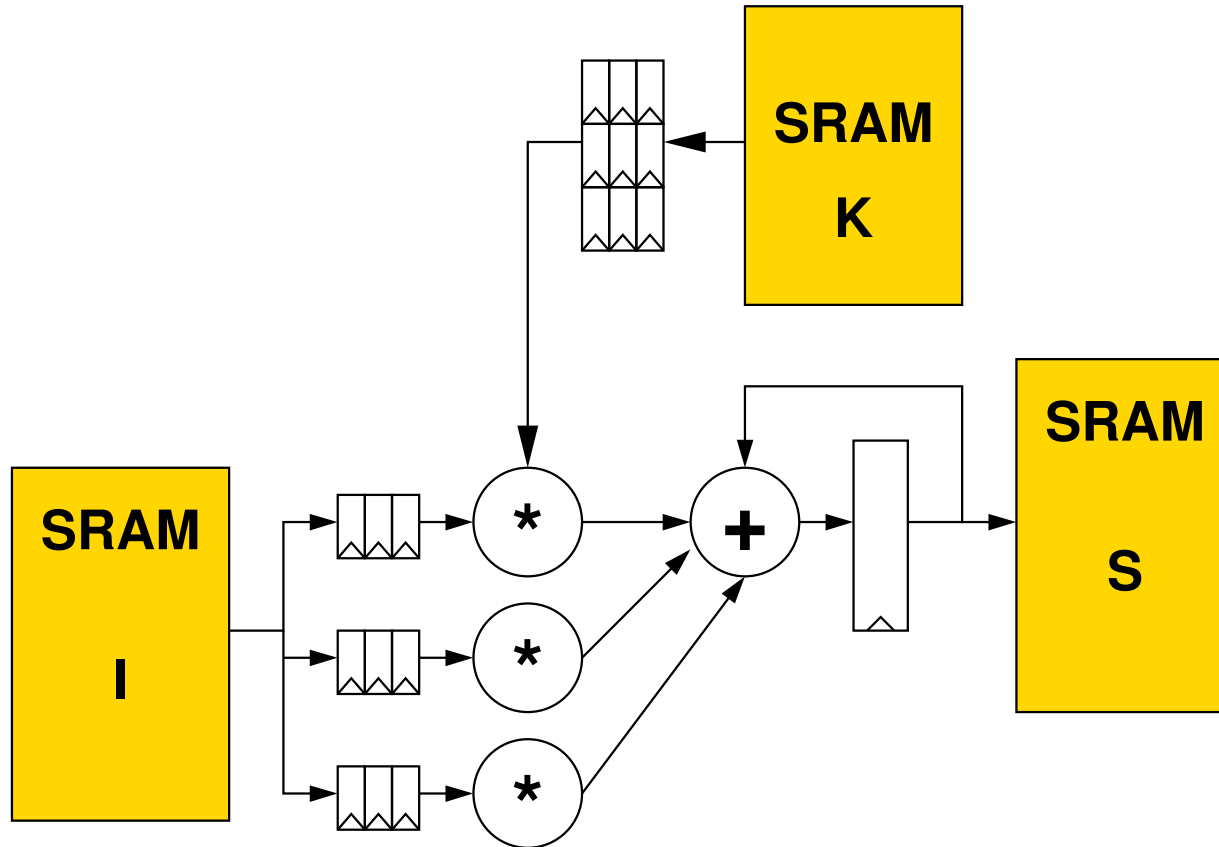
Deux mémoires



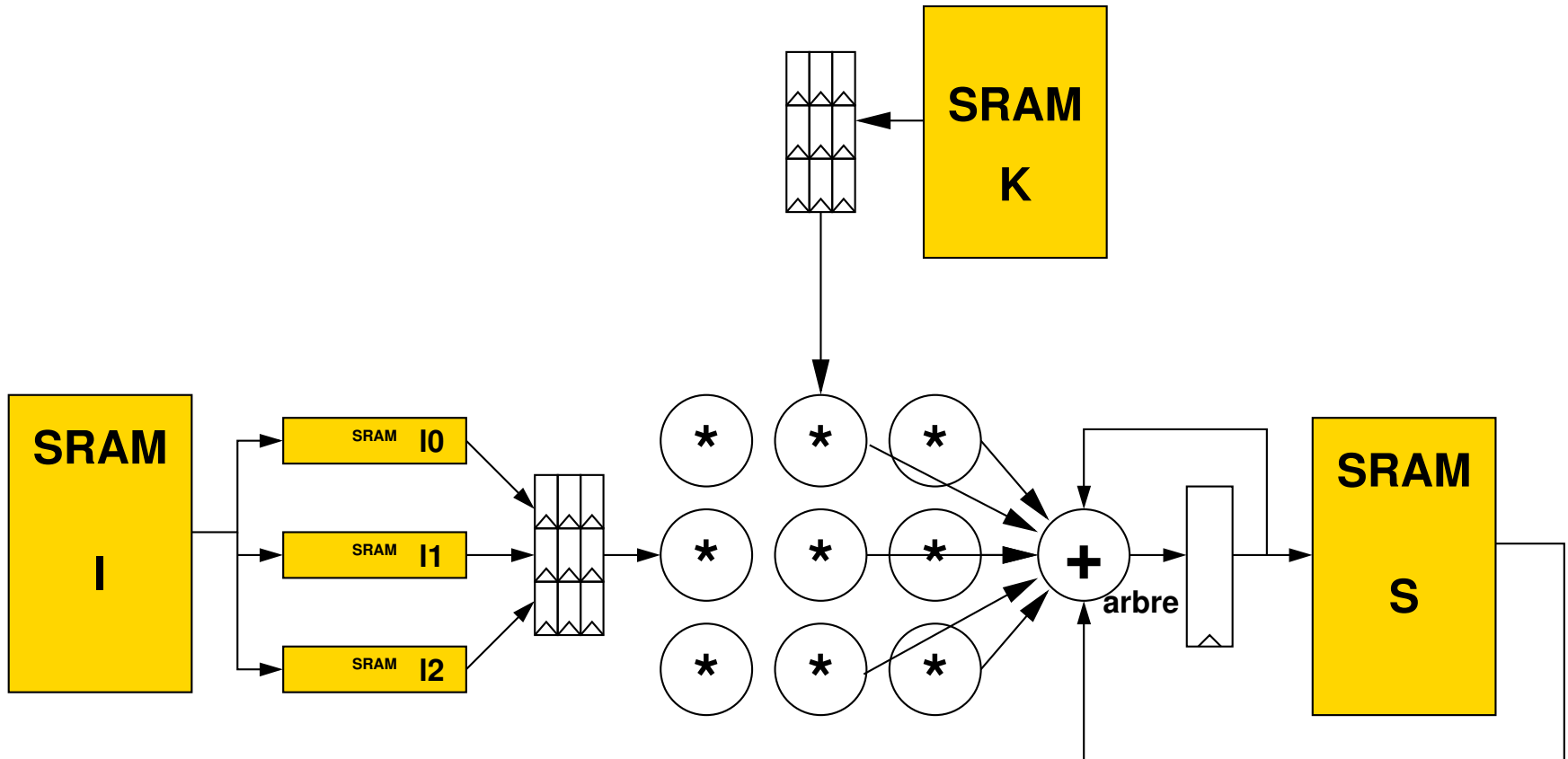
Trois mémoires



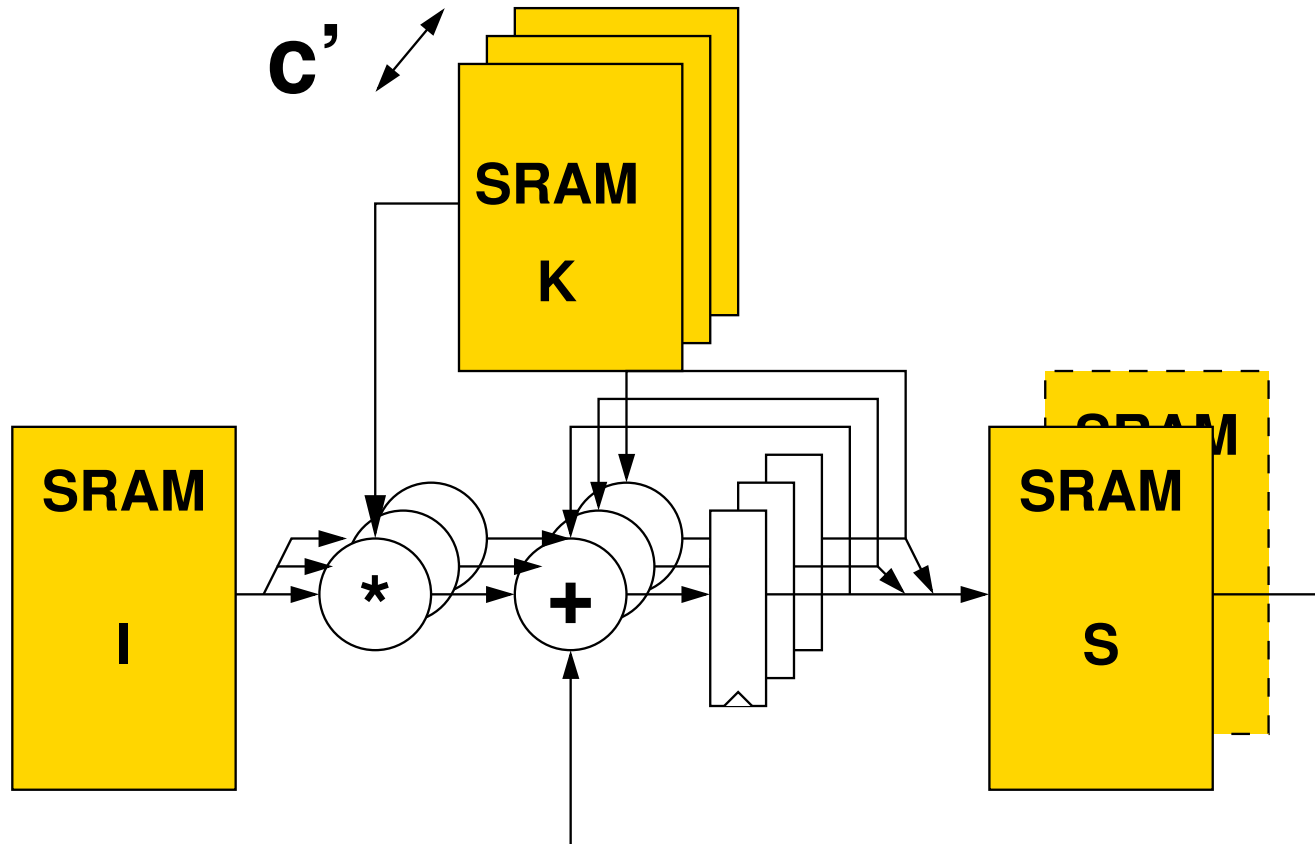
Trois mémoires, 3 multiplieurs



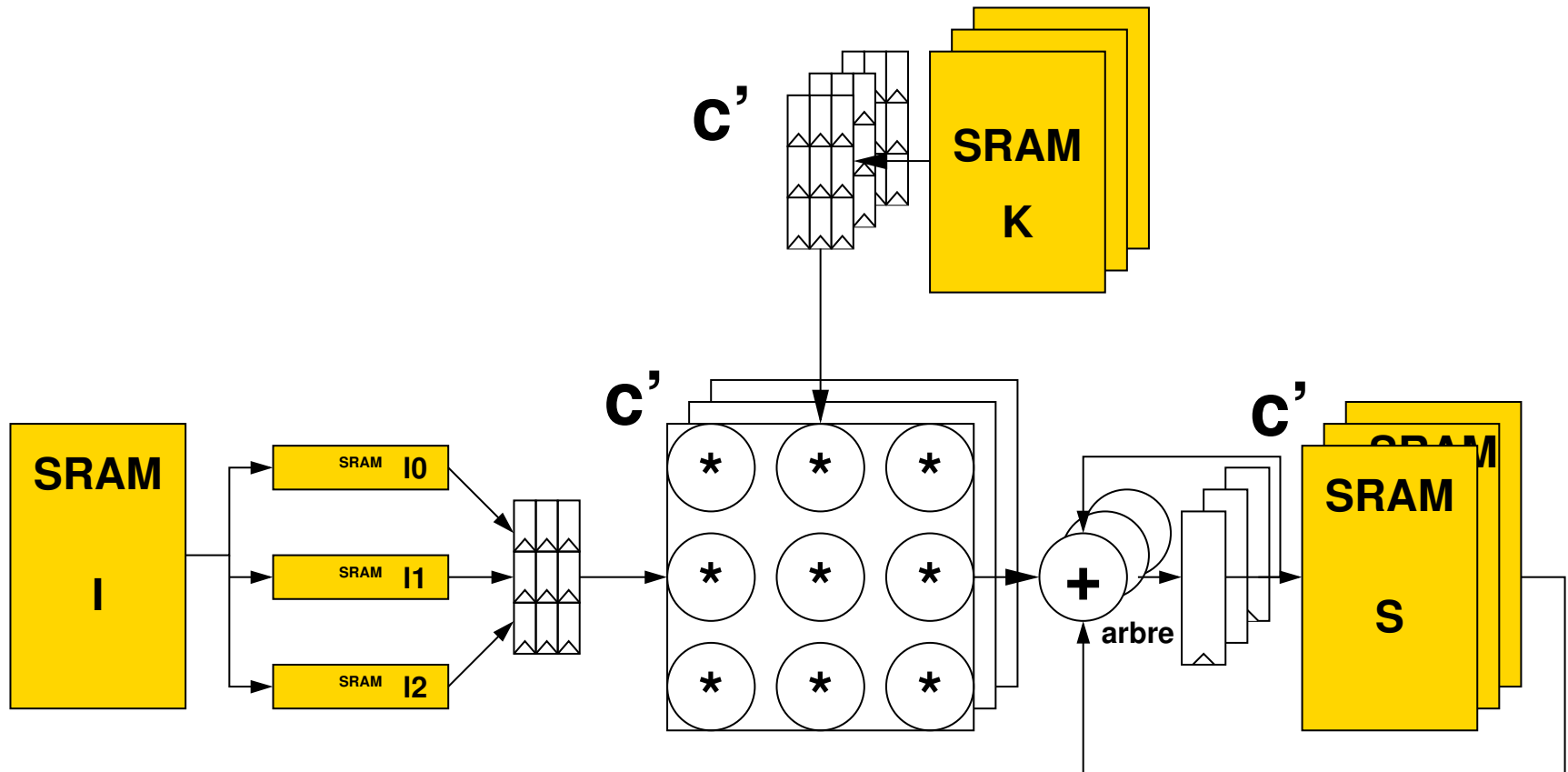
Trois mémoires, 9 multiplieurs



Parallélisation des kernels, mutualisation des entrées



Parallélisation des kernels, mutualisation des entrées multiples



Unification des Convolution et Perceptron

La convolution est un cas particulier de Perceptron totalement connecté.

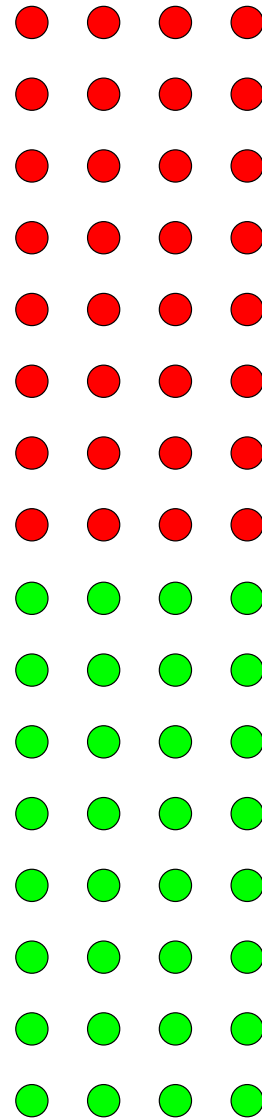
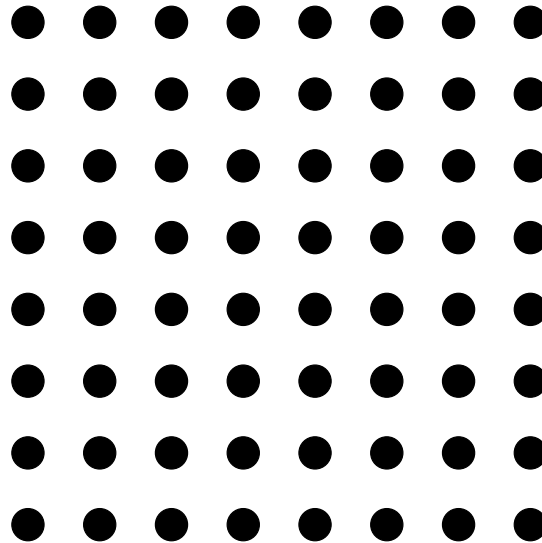
Dans certains cas, il peut être avantageux de la remettre sous forme de produit matriciel.

$$\begin{pmatrix} I_{-1,-1,0} & I_{0,-1,0} & I_{1,-1,0} & I_{-1,0,0} & I_{0,0,0} & I_{1,0,0} & \dots \\ I_{0,-1,0} & I_{1,-1,0} & I_{2,-1,0} & I_{0,0,0} & I_{1,0,0} & I_{2,0,0} & \dots \end{pmatrix} \begin{pmatrix} K_{0,0,0,0} & K_{0,0,0,1} & \dots \\ K_{1,0,0,0} & K_{1,0,0,1} & \dots \\ K_{2,0,0,0} & K_{2,0,0,1} & \dots \\ K_{0,1,0,0} & K_{0,1,0,1} & \dots \\ K_{0,1,0,0} & K_{0,1,0,1} & \dots \\ \vdots & & \end{pmatrix} = \begin{pmatrix} I_{-1,-1,0}K_{0,0,0,0} + I_{0,-1,0}K_{1,0,0,0} + I_{1,-1,0}K_{2,0,0,0} + I_{-1,0,0}K_{0,1,0,0} + \dots & \dots \\ I_{0,0,0}K_{0,0,0,0} + I_{1,0,0}K_{1,0,0,0} + I_{1,0,0}K_{2,0,0,0} + I_{1,0,0}K_{0,1,0,0} + \dots & \dots \end{pmatrix}$$

- Le produit Matrice-Vecteur du Perceptron devient un opérateur générique.
- Pour les CNNs, les matrices sont remplies par des fenêtres glissantes sur l'image.

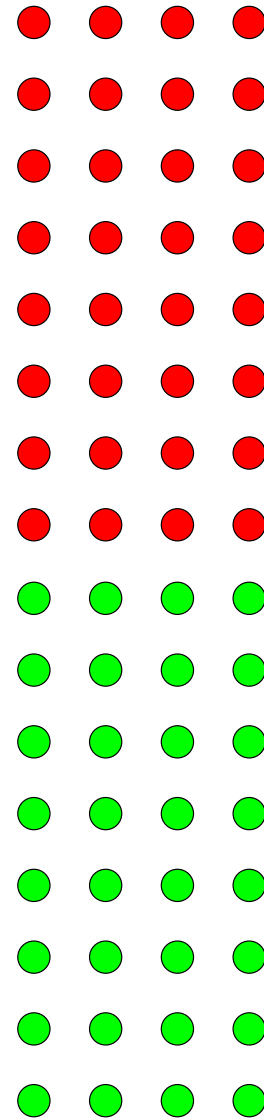
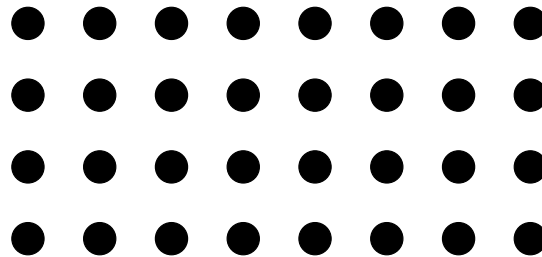
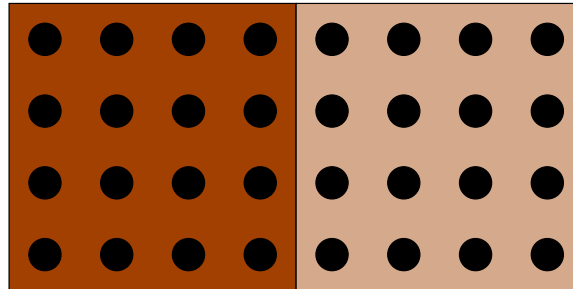
Décomposition du produit matriciel

$$\text{Green Square} = \text{Black Square} * \text{Red Square}$$

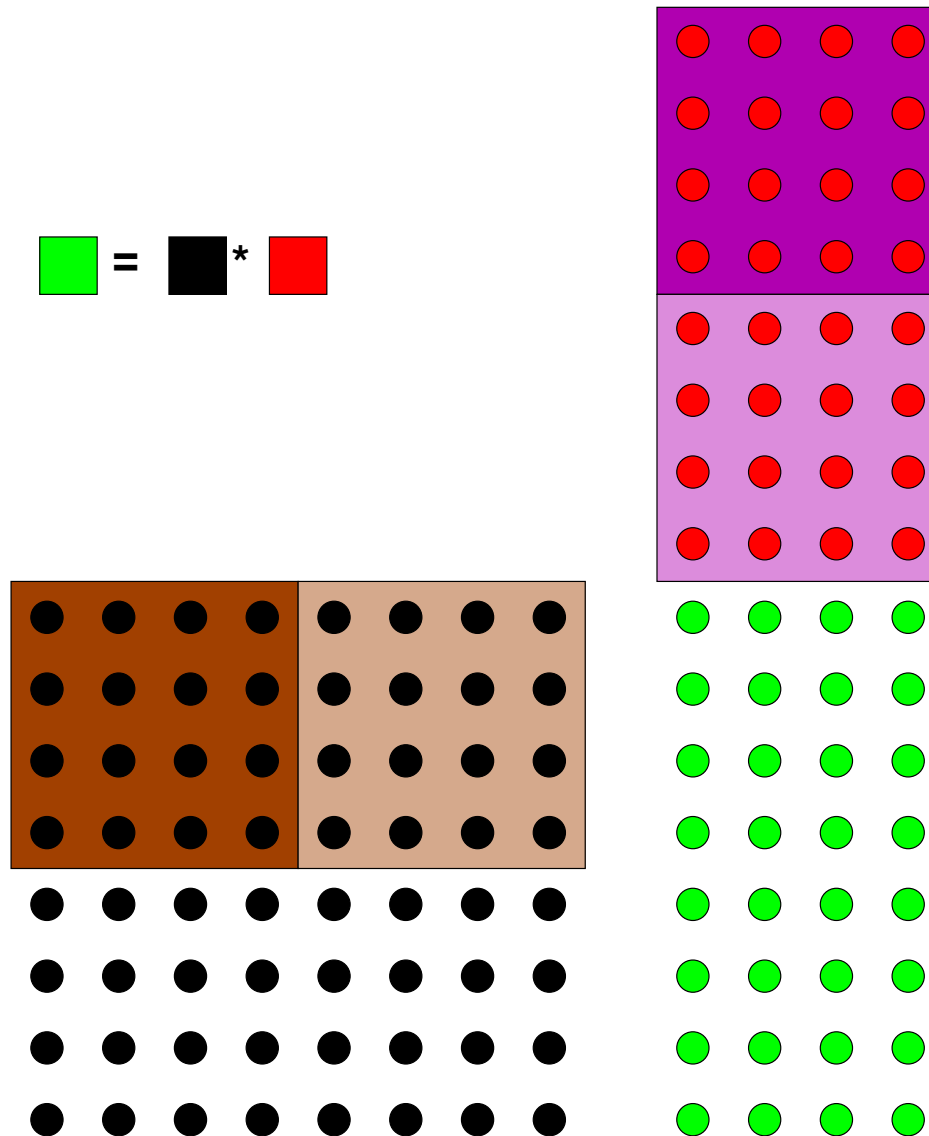


Décomposition du produit matriciel

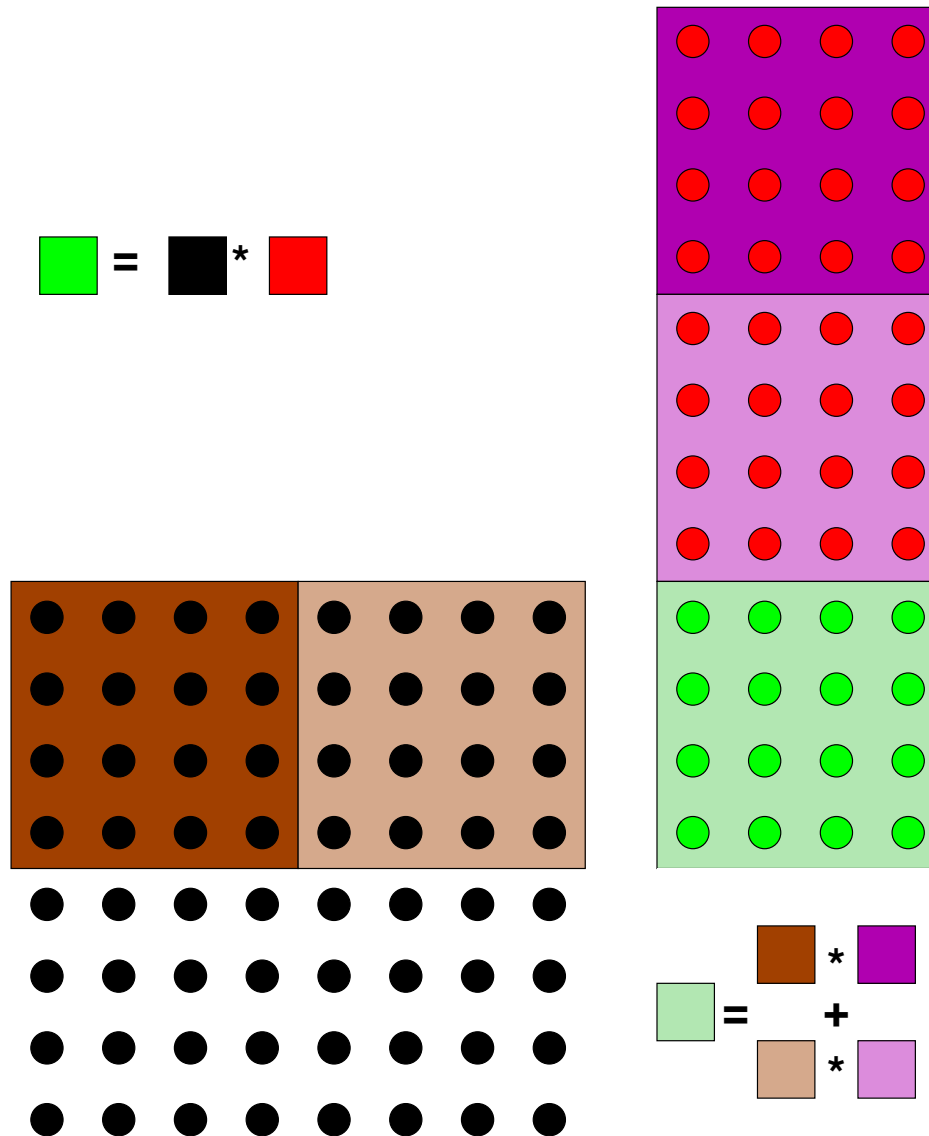
$$\text{Green Square} = \text{Black Square} * \text{Red Square}$$



Décomposition du produit matriciel



Décomposition du produit matriciel



CNN et produits matriciel

La représentation d'une convolution par un produit matriciel permet de décomposer le calcul en sous-produits matriciels câblés.

Les données intermédiaires sont alignées sur les tailles des opérateurs câblés.

- ☐ Il devient possible de faire des opérateurs matériels systématiquement ré-utilisés pour toutes les couches du CNN
- ☐ Les opérateurs de base sont optimisés au plus proche du matériel

Parallélisation et vectorisation

La **vectorisation** peut être associée à du **déroutement de boucle** total/partiel.

Le déroulement de boucle est une technique de parallélisation dans les boucles d'itérations indépendantes:

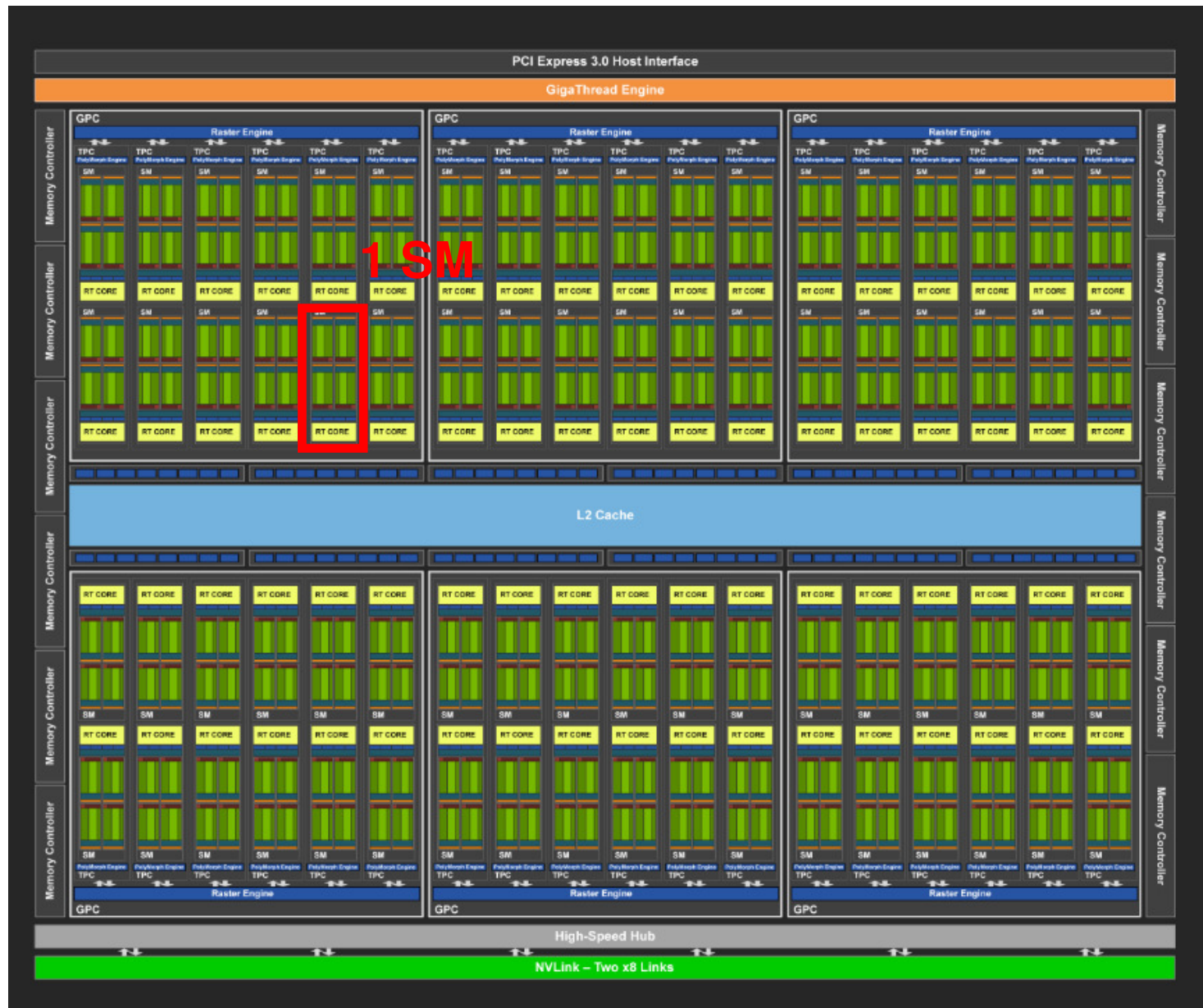
```
for i=0, i in [0,18[  for i=0, i in [0,18[, step 3
    calcul(i, data(i))      calcul(i, data(i))//
                           calcul(i+1, data(i+1))//
                           calcul(i+2, data(i+2))
```

- Les accès mémoire des calculs doivent être parallélisés:
-  Stockage des données dans différentes mémoires
-  Vectorisation/compactage des données dans de grands mots mémoire
- Et les données réparties sur les unités de calcul.
- Les résultats produits en parallèle doivent eux-même être distribués sur des mémoires.

Plan

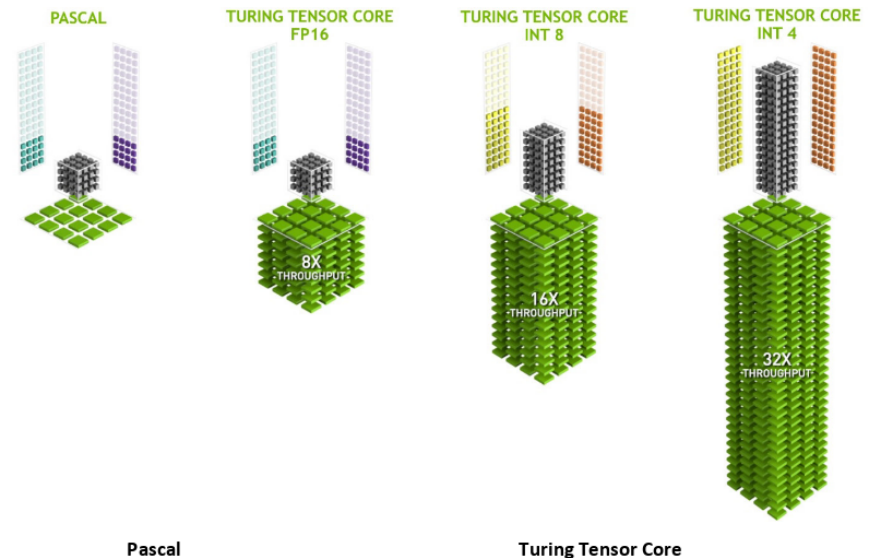
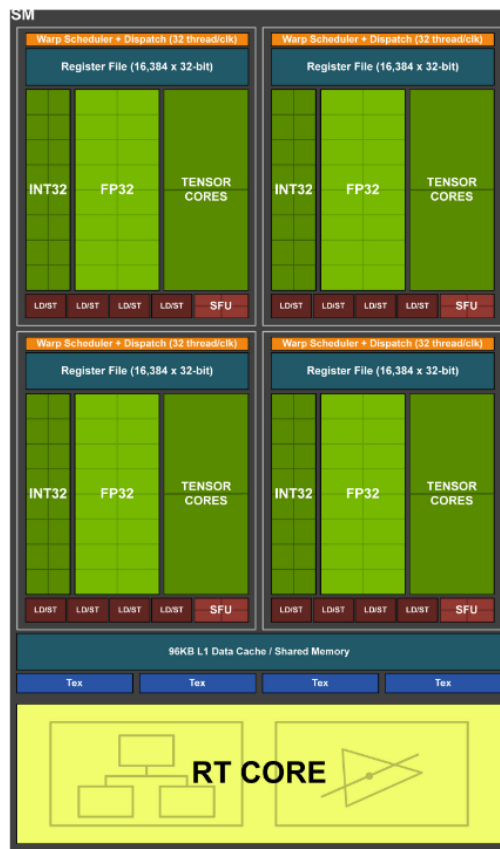
- ❑ Virgule fixe
- ❑ Réseaux de neurones
- ✖ Architecture pour les réseaux de neurones
 - ❑ CNN sur CPU
 - ❑ Architectures spécifiques
 - ✓ CNN & GPU
- ❑ Exemple d'architectures spécifiques

Nvidia Turing



Nvidia Turing SM & Tensor Core

A TU102 GPU contains 576 Tensor Cores: eight per SM and two per each processing block within an SM. Each Tensor Core can perform up to 64 floating point fused multiply-add (FMA) operations per clock using FP16 inputs. **Eight Tensor Cores in an SM** perform a total of 512 FP16 multiply and accumulate operations per clock, or 1024 total FP operations per clock. The new **INT8** precision mode works at double this rate, or **2048 integer operations per clock**.



Autres optimisations

❑ Arithmétique

Pour réduire la taille des opérateurs, la quasi-linéarité des réseaux de neurones permet d'utiliser la virgule fixe. Typiquement 8 à 16 bits conviennent.

Des stratégies extrêmes (binaire/ternaires) ont l'inconvénient de nécessiter plus de coefficients pour le même taux de détection (pb de codage de l'information) et ne sont pas assez précises.

❑ Algorithmique

- Exploiter l'**éparpillement** (sparsity) des coefficients, dont un grand nombre peut être fixé à 0.
- Compresser les réseaux de neurones
- Transformer les architectures de réseau (exploitation des redondances), mutualiser les calculs

👉 Les meilleures stratégies sont au niveau algorithmique.

Plan

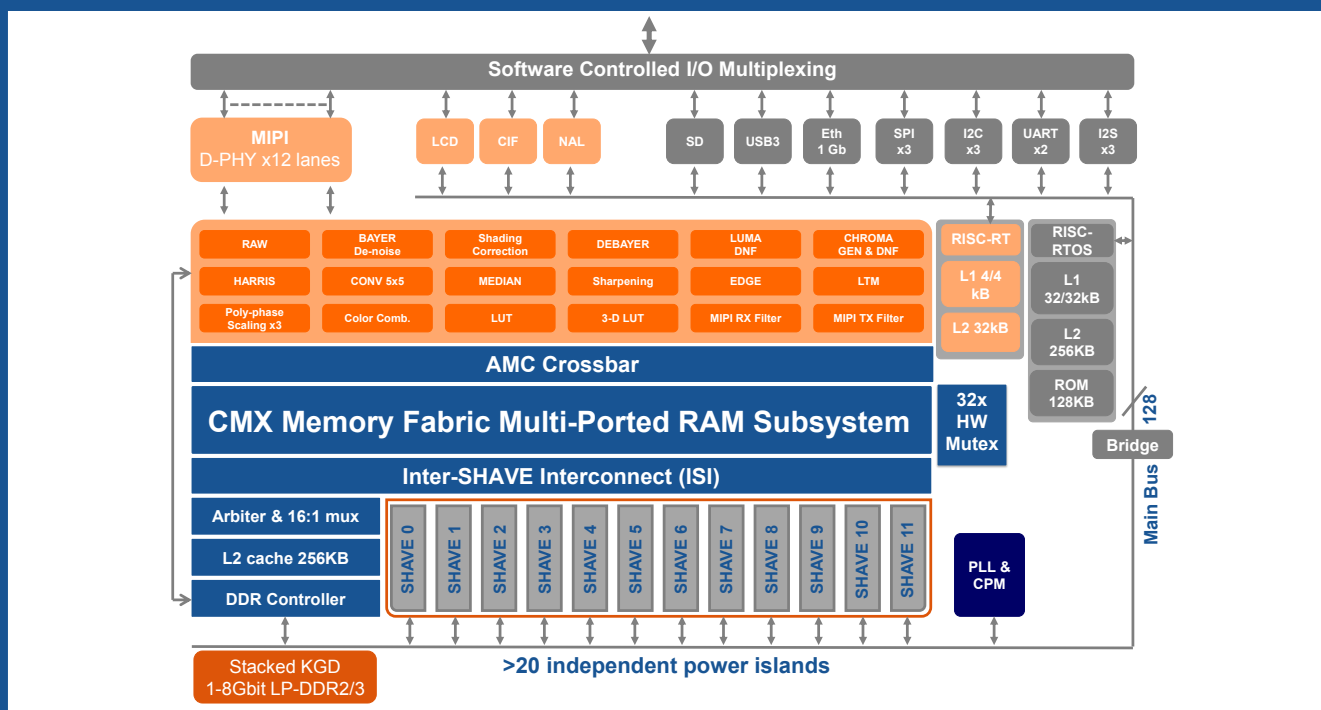
- ❑ Virgule fixe
- ❑ Réseaux de neurones
- ❑ Architecture pour les réseaux de neurones
- ✗ Exemple d'architectures spécifiques

Movidius (Intel)- architecture système

MYRIAD 2 MA2x50

12

Vision Processing Unit (VPU) Architecture



www.movidius.com
© Copyright Movidius 2016



Cambricon-X (Huawei)

Cambricon-X (Huawei) exploite la 'sparsité' des coefficients: les coefficients de valeur très faible sont mis à zéro. Hors ligne, les coefficients sont encodés (Run Length Encoders, Zéro Skipping, etc. . .).

A l'inférence, le matériel gère directement le décodage, au fur et à mesure des calculs.

Références : "Cambricon-X: An Accelerator for Sparse Neural Networks", Shijin Zhang & al., 2016

Cambricon-X (Huawei) - Principe

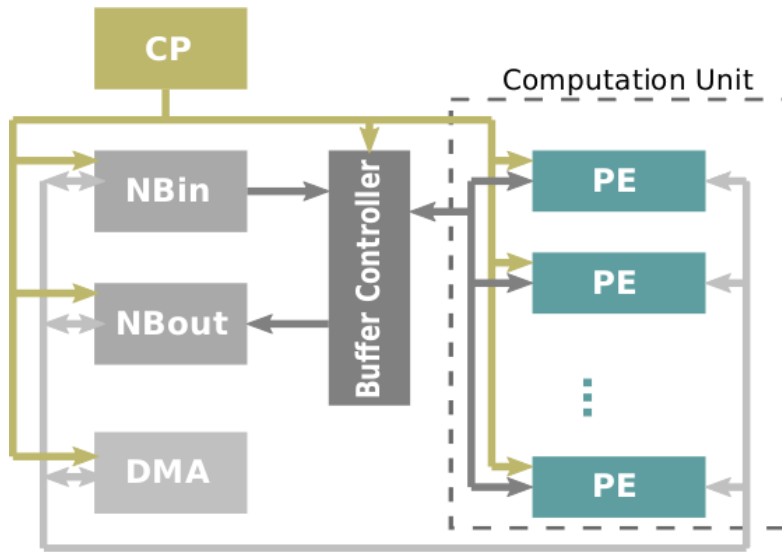


Fig. 4. Accelerator architecture.

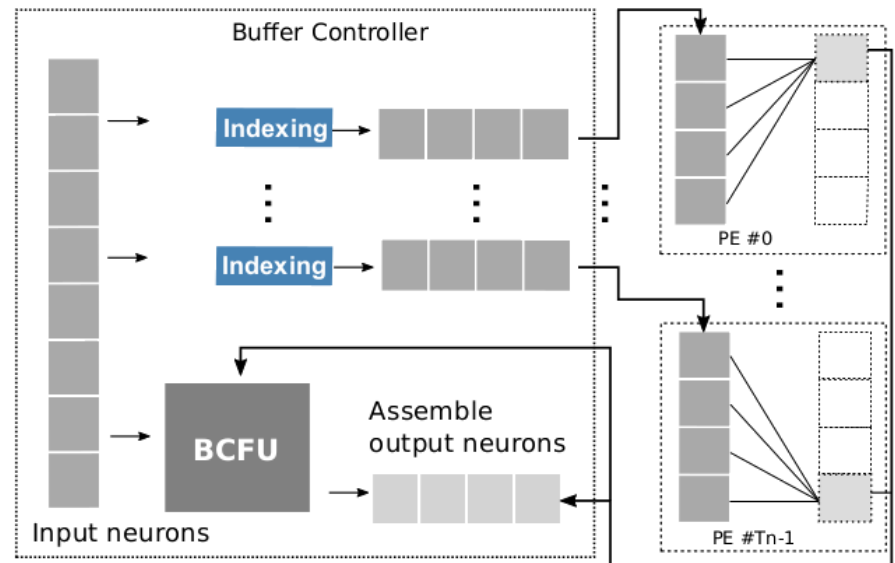


Fig. 5. Buffer controller architecture.

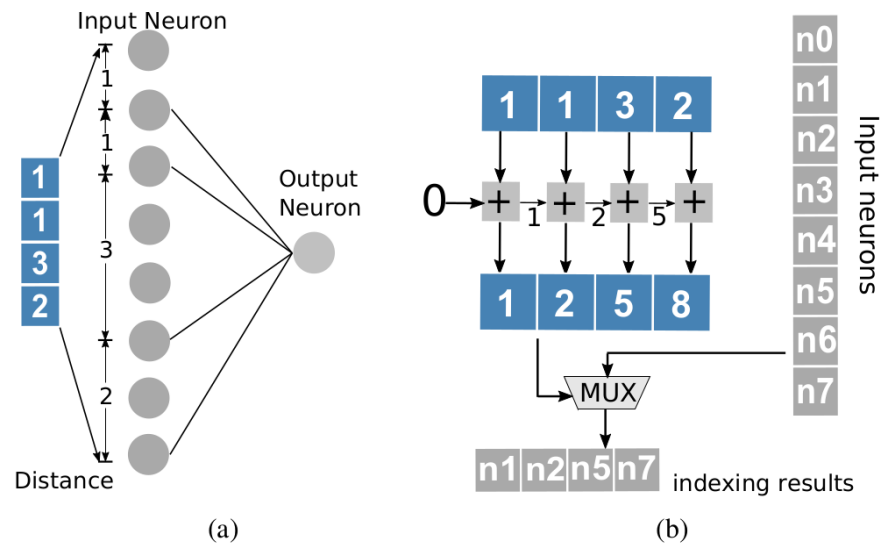


Fig. 11. (a) A sparse network example with the step indexing. (b) Hardware implementation of the step indexing.

Cambricon-X (Huawei) - Performance

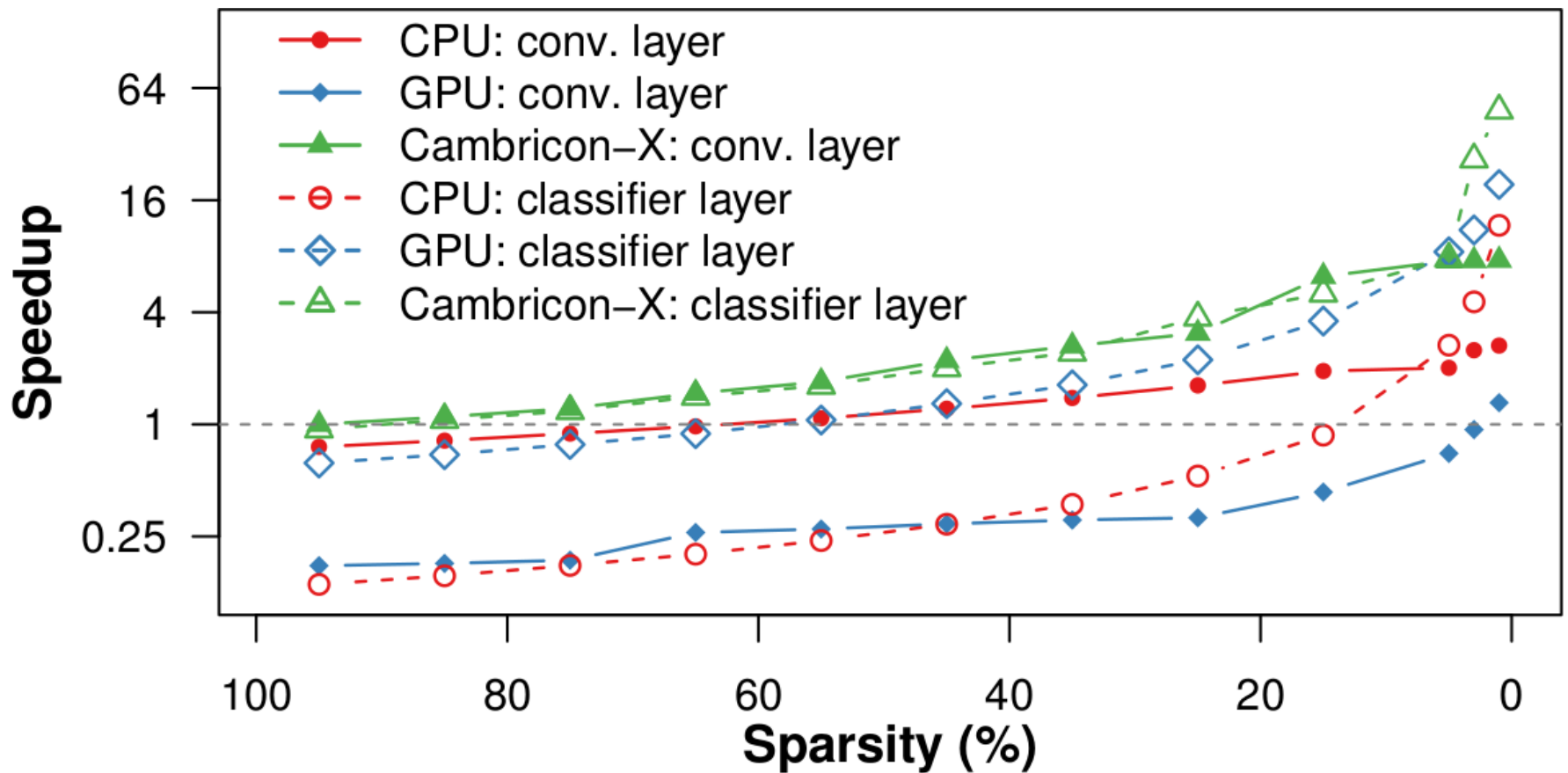


Fig. 19. Speedup of sparse layer over dense layer.

Afin de bénéficier de la 'sparsity', la (dé)compression a un coût, et seuls des opérateurs matériels l'exploitent pleinement.

➡ Version 'mobile' **2 TeraOPS FP16, 0.5 Watt**

➡ Version cartes PCI **83.2 TFLOPS-FP16 ou 166.4 TOPS-INT8, 110 W.**

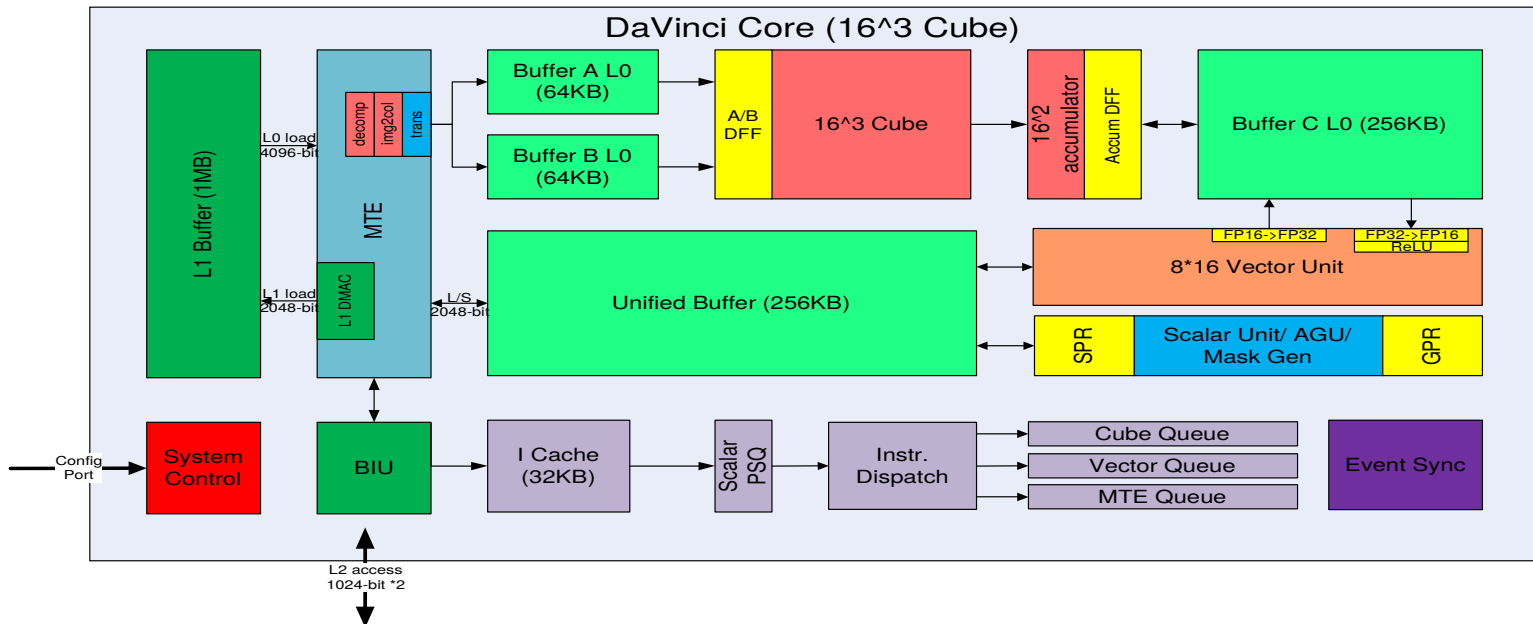
Ascend (Huawei)

La génération Ascend change de stratégie: le décodage est séparé du calcul de convolution.

- ❑ La gestion individuelle des données et coefficients pour le décodage a un coût trop élevé
- ❑ Il n'est pas possible de bénéficier de la décomposition de la multiplication

L'unité de convolution a une taille fixe (produit matriciel $16 \times 16 \times 16$) et le décodage des réseaux élagués (pruning) est suivi d'une décomposition matricielle.

DaVinci Core



- **Cube** : 4096(16³) FP16 MACs + 8192 INT8 MACs
- **Vector** : 2048bit INT8/FP16/FP32 vector with special functions (activation functions, NMS- Non Minimum Suppression, ROI, SORT)
- Explicit memory hierarchy design, managed by MTE

In its maximum configuration (16x16x16), the cube can perform 4,096 FP16 MAC operations per clock cycle. To support highly scalable memory, each

Da Vinci Core is equipped with dedicated SRAMs with fixed functions and variable capacities to accommodate different computing power scenarios.

➡ Performance: 16 TOPS-INT8 and 8 TOPS-FP16, Max Power: 8W

Conclusion

Il existe une très grande variété d'architectures, liées aux différents types d'applications (détection, reconnaissance, traitement. . .) et contextes (cloud, desktop, mobile, . . .).

Une architecture exploite une propriété spécifique, statistique, et bien souvent a une performance optimisée pour un un contexte.

➡ **Aucune architecture n'est la meilleure sur tous les aspects!**

Pour l'examen, vous étudierez un document d'archi pour l'IA, qui fera l'objet de quelques questions.

Architecture des SoC

Architectures pour les réseaux de neurones

S. Mancini

Plan Détaillé

✖ Virgule fixe

- ☆ Nombres entiers et Virgule flottante
- ☆ Problématique des opérations virgule flottante
- ☆ Opérateurs virgule flottante
- ☆ Opérateur matériels en virgule flottante
- ☆ Représentation virgule fixe
- ☆ Comparaison virgule fixe/flottante
- ☆ Usages
- ☆ Arrondi , troncature & saturation
- ☆ Addition
- ☆ Addition
- ☆ Multiplication
- ☆ Multiplication
- ☆ Division
- ☆ Notion de bruit de calcul
- ☆ Cycle de développement (D. Menard)
- ☆ Bilan sur la virgule fixe

✖ Réseaux de neurones

□ Réseaux de neurone et apprentissage

- ☆ Principe généraux de l'IA & réseaux de neurones
- ☆ Réseaux de Neurones & CNN
- ☆ Réseaux de Neurones & CNN
- ☆ Apprentissage
- ☆ Fonctions génériques 1/2
- ☆ Fonctions génériques 2/2
- ☆ L'apprentissage d'un réseau de neurones

□ Convolutional Neural Network (CNN)

- ☆ CNN
- ☆ Structures de données d'un CNN
- ☆ Convolution
- ☆ Convolution (bis)
- ☆ RELU
- ☆ MaxPool
- ☆ SoftMax
- ☆ Complètement connectée
- ☆ Exemples de CNN
- ☆ Complexité des CNN
- ☆ GoogleNet - Architecture

- ☆ GoogleNet - Puissance de calcul

✖ Architecture pour les réseaux de neurones

- ☆ Analyse pour l'architecture
- ☆ Parallélisation
- ☆ Parallélisation "haut niveau"
- CNN sur CPU
 - ☆ Vectorisation sur CPU
 - ☆ Vectorisation & CNN
 - ☆ Vectorisation et alignement
 - ☆ Vectorisation selon les canaux
 - ☆ Vectorisation selon les éléments
- Architectures spécifiques
 - ☆ Opportunités des architectures spécialisées
 - ☆ Architectures de calcul
 - ☆ FPGA/ASIC
 - ☆ FPGA/ASIC
 - ☆ FPGA/ASIC
 - ☆ FPGA/ASIC
 - ☆ FPGA/ASIC

- ☆ FPGA/ASIC
- ☆ FPGA/ASIC
- ☆ FPGA/ASIC
- ☆ Unification des Convolution et Perceptron
- ☆ Décomposition du produit matriciel
- ☆ CNN et produits matriciel
- ☆ Parallélisation et vectorisation

□ CNN & GPU

- ☆ Nvidia Turing
- ☆ Nvidia Turing SM & Tensor Core
- ☆ Autres optimisations

✖ Exemple d'architectures spécifiques

- ☆ Movidius (Intel)- architecture système
- ☆ Movidius (Intel) - unités vectorielles
- ☆ Cambricon-X (Huawei)
- ☆ Cambricon-X (Huawei) - Principe
- ☆ Cambricon-X (Huawei) - Performance
- ☆ Ascend (Huawei)
- ☆ Ascend (Huawei)
- ☆ Conclusion